



Escuela
Politécnica
Superior

Resolución de problemas de la Olimpiada Lingüística mediante modelos de lenguaje de gran escala



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autora:

Alicja Wiktorja Hyska

Tutor:

Juan Antonio Pérez Ortiz

Julio 2026



Universitat d'Alacant
Universidad de Alicante

Resolución de problemas de la Olimpiada Lingüística mediante modelos de lenguaje de gran escala

Aplicación de modelos de inteligencia artificial

Autora

Alicja Wiktoria Hyska

Tutor

Juan Antonio Pérez Ortiz

Lenguajes y Sistemas Informáticos



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Julio 2026

Preámbulo

Este trabajo surge del interés por explorar los límites del razonamiento lingüístico en los modelos de lenguaje actuales. El objetivo es diseñar y evaluar estrategias de prompting que permitan a estos modelos resolver puzzles de olimpiadas lingüísticas a partir de un conjunto reducido de ejemplos y sin recibir información explícita previa sobre la lengua analizada. El trabajo se enmarca en el área de procesamiento de lenguas de baja densidad de recursos y ha sido realizado bajo la supervisión del profesor Juan Antonio Pérez Ortiz en el Grado de Ingeniería Informática de la Universidad de Alicante.

Agradecimientos

Quisiera agradecer, en primer lugar, a mi tutor, Juan Antonio Pérez Ortiz, por su orientación, sus consejos y su disponibilidad durante el desarrollo de este Trabajo de Fin de Grado. Su ayuda ha sido fundamental para dar forma al proyecto y para afrontar las distintas decisiones que han ido surgiendo a lo largo del proceso.

También quiero dar las gracias a mi familia, por su apoyo constante durante todos estos años. Su confianza, paciencia y ánimo han sido esenciales no solo durante la realización de este trabajo, sino a lo largo de toda mi etapa universitaria.

Este trabajo nace también de un interés personal por los idiomas y por la forma en que las lenguas permiten entender distintas culturas. Haber vivido en diferentes países me ha permitido acercarme a varios idiomas desde una perspectiva práctica y humana, y ha despertado en mí una curiosidad especial por la traducción y el razonamiento entre lenguas. Esa experiencia personal ha influido directamente en la elección de este tema, centrado en la resolución de problemas de olimpiadas lingüísticas mediante modelos de inteligencia artificial.

Por último, quiero agradecer a todas las personas que, de una forma u otra, me han acompañado durante este camino: compañeros, amigos y profesores que han contribuido a mi formación y me han ayudado a llegar hasta aquí.

El límite de mi lenguaje es el límite de mi mundo.

Ludwig Wittgenstein.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Contexto y estado del problema	2
1.3. Estructura del documento	2
2. Marco teórico y estado de la cuestión	5
2.1. Lenguas de baja densidad de recursos	5
2.2. Razonamiento metalingüístico	6
2.3. Benchmarks de olimpiadas lingüísticas para LLMs	6
2.3.1. PuzzLing Machines	7
2.3.2. modeLing	7
2.3.3. Linguini	8
2.3.4. LingOly	9
2.3.5. Comparación entre los principales benchmarks	10
2.4. Estrategias de prompting para razonamiento lingüístico	10
2.4.1. Prompting analógico	11
2.4.2. Gramática en contexto	11
2.4.3. Aprendizaje en contexto con ejemplos	12
2.4.4. Razonamiento paso a paso	12
2.4.5. Autocorrección y revisión de respuestas	13
2.5. Análisis de las dificultades de los LLMs en puzzles lingüísticos	13
2.6. Síntesis y posicionamiento del trabajo	15
3. Objetivos	17
3.1. Objetivo principal	17
3.2. Objetivos específicos	17
3.3. Restricciones y condicionantes	18
4. Metodología y datos	21
4.1. Dataset: Linguini	21
4.1.1. Criterios de selección	21
4.1.2. Puzzles seleccionados	22
4.1.3. Formato de los datos	22
4.2. Modelos evaluados	22
4.3. Estrategias de prompting	23
4.3.1. Instrucción metalingüística base	23
4.3.2. M0: Control exploratorio previo	23
4.3.3. M1: Baseline	24
4.3.4. M2: Presentación incremental acumulativa	25

4.3.5.	M2.2: Exploración combinatoria por tamaños crecientes	26
4.3.6.	M3: Step-by-step lingüístico	27
4.3.7.	M4: Inspiración humana	28
4.3.8.	M5: Autocorrección iterativa	29
4.4.	Sistema experimental	30
4.5.	Métricas de evaluación	30
4.6.	Análisis cualitativo	32
4.6.1.	Proceso de resolución humana	32
4.6.2.	Comparación con el modelo	35
5.	Desarrollo del sistema	37
5.1.	Arquitectura general	37
5.1.1.	Formato de entrada: el puzzle	37
5.1.2.	Formato del log crudo	38
5.1.3.	Formato del JSON de métricas	39
5.2.	Diseño de los prompts	40
5.2.1.	Instrucción metalingüística base	41
5.2.2.	Bloque dinámico de traducciones	41
5.2.3.	Prompt de M0: verificación previa	42
5.2.4.	Prompt de M1: baseline	42
5.2.5.	Prompt de M2: presentación incremental acumulativa	43
5.2.6.	Prompt de M3: step-by-step lingüístico	44
5.2.7.	Prompt de M4: inspiración humana	45
5.2.8.	Prompt de M5: segundo revisor	47
5.3.	Extracción automática de traducciones	48
5.3.1.	Algoritmo de extracción	48
5.3.2.	Gestión de respuestas incompletas	49
5.3.3.	Detección de bucles de repetición	50
5.4.	Problemas encontrados y soluciones	50
5.4.1.	Limitaciones de las APIs	50
5.4.2.	Problemas de extracción y métricas	51
5.4.3.	Otras limitaciones	51
6.	Resultados	53
6.1.	M0 — Verificación previa	53
6.2.	M1 — Baseline	54
6.3.	M2 — Incremental acumulativo	56
6.4.	M3 — Step-by-step lingüístico	57
6.5.	M4 — Inspiración humana	58
6.5.1.	Comparación cualitativa con la resolución humana	59
6.6.	M5 — Segundo revisor	60
6.7.	Experimentos complementarios	61
6.7.1.	M2.2 — Exploración combinatoria por tamaños crecientes	61
6.7.2.	Dirección inversa: inglés → lengua desconocida	62
6.7.3.	Tareas morfológicas y numéricas	64
6.8.	Resumen y análisis	66

7. Conclusiones	69
8. Trabajos futuros	71
8.1. Ampliación del conjunto de puzzles	71
8.2. Ampliación de evaluación de la dirección inversa y exploración combinatoria de ejemplos	71
8.3. Mejora de las estrategias de revisión	72
8.4. Más protocolos de razonamiento humano	72
8.5. Evaluación con nuevos modelos	72
8.6. Uso de salidas estructuradas	73
Bibliografía	75
A. Repositorio del proyecto	79

1. Introducción

1.1. Motivación

El lenguaje es una de las capacidades más complejas y distintivas de la especie humana. En el mundo existen miles de lenguas vivas (Eberhard y cols., 2020), pero la gran mayoría dispone de una presencia digital muy reducida, en muchos casos apenas existen corpus textuales y, por consiguiente, herramientas de procesamiento automático. Esta escasez de información hace que los sistemas de inteligencia artificial que dependen de grandes volúmenes de datos de entrenamiento, tengan muchas dificultades para trabajar con ellas, al menos de forma fiable.

Sin embargo, las personas somos capaces de aprender a leer, escribir y traducir en un idioma completamente desconocido a partir de un número reducido de ejemplos, gracias a nuestra *competencia metalingüística*. En otras palabras, podemos identificar el vocabulario, reconocer patrones morfológicos y formular reglas gramaticales únicamente comparando palabras y frases. Esta capacidad más característica y propia de los seres humanos plantea una pregunta fundamental para la inteligencia artificial: ¿pueden los modelos de lenguaje de gran escala hacer lo mismo?

La Olimpiada Internacional de Lingüística (IOL) propone anualmente problemas del tipo *piedra Rosetta*: se presenta al participante un conjunto de frases en una lengua completamente desconocida junto con sus traducciones, y se le pide que deduzca las reglas del idioma y las aplique para traducir nuevas frases (Derzhanski y Payne, 2010; Bozhanov y Derzhanski, 2013; International Linguistics Olympiad, 2026). Este tipo de problemas son autocontenidos, es decir, toda la información necesaria se encuentra en el enunciado, y requieren razonamiento explícito, no memorización. Por la forma en la que se presenta la información, los ejercicios de las olimpiadas lingüísticas presentan un banco de pruebas ideal para medir si un sistema de inteligencia artificial posee razonamiento metalingüístico genuino, o al menos si es capaz de reproducir este tipo de razonamiento (Şahin y cols., 2020; Sánchez y cols., 2024).

En este trabajo vamos a partir de la hipótesis de que es posible diseñar estrategias de prompting que permitan a los modelos de lenguaje resolver estos problemas de forma progresiva, construyendo explícitamente un diccionario y un conjunto de reglas gramaticales a partir de los ejemplos proporcionados, sin recibir ninguna pista sobre la lengua en cuestión. En el caso de que la hipótesis se confirme, esto abriría la puerta a utilizar modelos de lenguaje para procesar, documentar y analizar miles de lenguas del mundo para las que no existe ni existirá suficiente cantidad de datos de entrenamiento. En caso contrario, estudiaremos y evidenciaremos hasta qué punto los modelos actuales pueden reproducir un proceso de análisis similar al que sigue una persona al enfrentarse a una lengua que no conoce.

Cabe mencionar que la motivación del proyecto también está relacionada con mi interés personal por los idiomas y por la forma en que distintas lenguas expresan una misma realidad mediante formas tan diferentes.

1.2. Contexto y estado del problema

Durante los últimos años, los modelos de lenguaje de gran escala han demostrado capacidades notables en tareas como la generación de texto, la traducción automática o la respuesta a preguntas (Brown y cols., 2020). Una de sus características más relevantes es el aprendizaje en contexto, que es la posibilidad de realizar una tarea a partir de ejemplos incluidos directamente en el prompt, es decir, en la forma en la que se le presenta la información al modelo mediante la petición, sin necesidad de modificar sus parámetros. Aquí vamos a plantear una pregunta especialmente relevante para las lenguas de baja densidad de recursos: ¿puede un modelo deducir patrones lingüísticos a partir de unos pocos ejemplos, incluso cuando no dispone de información suficiente para apoyarse únicamente en los datos aprendidos durante su entrenamiento?

En los últimos años se han propuesto varios conjuntos de evaluación basados en problemas de olimpiadas lingüísticas, de los que hablaremos con más detalle en el Capítulo 2. Estos datasets reúnen problemas de distintas lenguas y permiten evaluar la capacidad de los modelos para resolver tareas como la identificación de patrones morfológicos y la traducción de textos (Şahin y cols., 2020; Chi y cols., 2024; Sánchez y cols., 2024; Bean y cols., 2024). Los resultados publicados hasta el momento muestran que estos problemas siguen siendo difíciles para los modelos actuales. Aunque algunos sistemas obtienen respuestas parcialmente correctas, su rendimiento se mantiene por debajo del nivel humano en muchos tipos de ejercicios, especialmente cuando las lenguas presentan una morfología compleja o estructuras alejadas de las más frecuentes en los datos de entrenamiento.

Distintos trabajos han estudiado formas de mejorar estos resultados mediante estrategias de prompting más elaboradas. En lugar de solicitar directamente una traducción final, estas estrategias guían al modelo para que identifique vocabulario, deduzca reglas y revise sus hipótesis antes de responder (Wei y cols., 2022; Zhu y cols., 2025; Madaan y cols., 2023). Precisamente de esto trata este trabajo de fin de grado. El objetivo es diseñar, implementar y comparar varias estrategias de prompting para la resolución de problemas de olimpiadas lingüísticas. En particular, se estudiarán distintas maneras de presentación de los ejemplos al modelo, que a su vez puedan permitir construir conocimiento de forma progresiva y corregir hipótesis incorrectas a medida que recibe nueva información.

1.3. Estructura del documento

El presente documento se organiza en los siguientes capítulos:

El **Capítulo 2** presenta el estado de la cuestión, revisando los fundamentos teóricos del razonamiento metalingüístico, los principales benchmarks de olimpiadas lingüísticas para LLMs y las estrategias de prompting más relevantes de la literatura.

El **Capítulo 3** define el objetivo principal del trabajo y los objetivos específicos que se derivan de él, así como las restricciones y condicionantes del proyecto.

El **Capítulo 4** describe la metodología del trabajo: los puzzles utilizados, los modelos evaluados, el control de dataset previo, las estrategias de prompting diseñadas, el sistema experimental implementado y las métricas de evaluación empleadas.

El **Capítulo 5** detalla la implementación del sistema experimental y las decisiones adoptadas durante su desarrollo.

El **Capítulo 6** presenta y analiza los resultados experimentales obtenidos, incluyendo la comparación con los valores de referencia del benchmark LINGUINI (Sánchez y cols., 2024) y el análisis cualitativo comparando el razonamiento del modelo con el humano. Se incluyen pruebas adicionales y complementarias que ayuden a evaluar el comportamiento del modelo para diferentes tipos de problemas.

El **Capítulo 7** recoge las conclusiones principales del trabajo, destacando los resultados más relevantes, las aportaciones realizadas y las limitaciones observadas durante la evaluación.

Finalmente, el **Capítulo 8** plantea las líneas de trabajo futuro que se abren a partir de los resultados obtenidos.

2. Marco teórico y estado de la cuestión

Este capítulo revisa el estado del arte en los campos que confluyen en este trabajo. En primer lugar, se describe la disponibilidad de recursos digitales de las lenguas del mundo y sus consecuencias para el procesamiento del lenguaje natural. A continuación, se introduce el concepto de razonamiento metalingüístico y su relación con los problemas de olimpiadas lingüísticas. Finalmente, se revisan los principales conjuntos de evaluación y estrategias de prompting utilizados para estudiar este tipo de razonamiento en modelos de lenguaje de gran escala.

2.1. Lenguas de baja densidad de recursos

Se estima que existen entre 6.000 y 7.000 lenguas vivas en el mundo (Eberhard y cols., 2020). Sin embargo, los lenguajes que todos conocemos o nos puedan sonar no tienen la misma presencia digital que otros de los que puede haber escasa información. Mientras que unas lenguas disponen de grandes cantidades de texto, corpus paralelos, diccionarios, herramientas de análisis y sistemas de traducción relativamente desarrollados, otras, apenas cuentan con material escrito accesible o con datos digitalizados suficientes para entrenar modelos de procesamiento del lenguaje natural.

En este contexto se habla habitualmente de lenguas de baja densidad de recursos (*low-resource languages*), y es que la disponibilidad de recursos puede variar mucho de una lengua a otra. Una lengua puede tener textos sin anotar, pero carecer de corpus paralelos, mientras que otra puede disponer de una gramática o de un diccionario elaborado por lingüistas, pero no de datos en formato útil para entrenar modelos y otras apenas tienen presencia escrita o digital. En todos los casos, el problema de fondo es el mismo: no existe suficiente material lingüístico disponible para aplicar directamente los métodos habituales basados en grandes volúmenes de datos.

Joshi y cols. (2020) cuantifican esta desigualdad mostrando que más del 88% de las lenguas del mundo apenas tienen representación en los conjuntos de datos utilizados para entrenar los modelos actuales. Esta falta de representación afecta de forma directa a tareas como la traducción automática, el reconocimiento de voz, el análisis morfosintáctico o la generación de texto. Los modelos actuales tienden a funcionar mejor en lenguas con mucha presencia digital, mientras que su rendimiento disminuye cuando se enfrentan a lenguas con pocos datos disponibles o con estructuras alejadas de las lenguas dominantes en el preentrenamiento. No sería lo mismo pedirle a un modelo que traduzca idiomas con gran cantidad de información digital como el inglés o el chino, que mostrarle una lengua sinotibetana únicamente hablada por pequeños grupos tribales como es el hakhun. Por ello, el problema no es solo técnico. La falta de herramientas para estas lenguas también afecta a su documentación, a su uso en entornos digitales y a la preservación del patrimonio lingüístico y cultural de sus comunidades hablantes. Por este motivo, el trabajo con lenguas de baja densidad de recursos es una línea

importante dentro del campo de procesamiento de lenguaje natural actual.

La solución viable para abordar este problema es el aprendizaje en contexto (*in-context learning*). En lugar de reentrenar un modelo con grandes cantidades de datos, se le proporcionan unos pocos ejemplos dentro del propio prompt (petición al modelo) y se observa si es capaz de deducir patrones a partir de ellos. Esta idea encaja especialmente bien con los problemas de olimpiadas lingüísticas, donde el objetivo no es memorizar una lengua, sino inferir reglas a partir de un conjunto reducido de ejemplos.

2.2. Razonamiento metalingüístico

El razonamiento metalingüístico consiste en observar una lengua como objeto de análisis. No se trata únicamente de usar el lenguaje para comunicarse, sino de fijarse en cómo funciona. Por ejemplo, ver qué elementos se repiten, qué partes de una palabra cambian, cómo se expresan el tiempo, la persona o el número, o qué orden siguen los constituyentes dentro de una frase.

Este tipo de razonamiento aparece de forma clara en los problemas de la Olimpiada Internacional de Lingüística (IOL), una competición internacional dirigida a estudiantes de secundaria en la que se plantean problemas de análisis lingüístico sobre lenguas normalmente desconocidas para los participantes (Derzhanski y Payne, 2010; International Linguistics Olympiad, 2026). En ellos, el participante recibe un conjunto reducido de frases en una lengua que no conoce, junto con sus traducciones. A partir de esa información debe deducir vocabulario, identificar patrones gramaticales y aplicar las reglas inferidas a nuevas frases. Estos problemas, como ya se ha mencionado, son autocontenidos: toda la información necesaria para resolverlos está en el propio enunciado. Por ello, este formato resulta especialmente útil para evaluar modelos de lenguaje de gran escala, ya que obliga a trabajar con pocos datos y a razonar sobre regularidades lingüísticas. La tarea no consiste únicamente en producir una traducción que parezca plausible, sino en mantener hipótesis coherentes y aplicarlas de forma sistemática. Por ejemplo, si el modelo identifica que una partícula marca pasado, debería aplicar esa hipótesis a todas las frases compatibles, y no solo a una de ellas. Esto se aplicaría tanto a los tiempos verbales como a otros rasgos gramaticales, como el género, el número, la conjugación verbal o incluso la diferencia entre una pregunta y una oración.

Aun así, es importante distinguir entre el resultado que observamos y el proceso interno del modelo. Que un modelo acierte una traducción no implica necesariamente que haya razonado de la misma forma que una persona. Puede haber aprovechado asociaciones parciales, patrones aprendidos durante el preentrenamiento o simplemente haber generado una respuesta plausible que se asemeja a la solución real. Por eso, en este proyecto nos interesa analizar también el diccionario, las reglas y las explicaciones que el modelo produce durante la resolución.

2.3. Benchmarks de olimpiadas lingüísticas para LLMs

En los últimos años han aparecido varios conjuntos de evaluación que utilizan problemas de olimpiadas lingüísticas para estudiar el razonamiento lingüístico de los modelos de lenguaje. Todos ellos parten de una idea parecida: proporcionar al modelo unos pocos ejemplos en una lengua poco conocida y comprobar si es capaz de extraer regularidades, formular hipótesis y

aplicarlas a nuevas expresiones. Este tipo de evaluación es nos sirve para estudiar el aprendizaje en contexto: a diferencia de otras tareas de traducción o comprensión lingüística, aquí no interesa medir únicamente cuánto sabe previamente el modelo sobre una lengua concreta, sino hasta qué punto puede razonar a partir de la información incluida en el propio enunciado. Por este motivo, estos benchmarks son una referencia directa para el diseño experimental de este trabajo.

A continuación se revisan los principales conjuntos de datos relacionados con esta línea de investigación.

2.3.1. PuzzLing Machines

El trabajo de Şahin y cols. (2020) es uno de los primeros intentos de evaluar sistemas automáticos mediante problemas de olimpiadas lingüísticas. Los autores recopilan un conjunto cercano a cien problemas de tipo piedra Rosetta, procedentes de olimpiadas lingüísticas, que cubren 81 lenguas pertenecientes a 39 familias distintas. A partir de este conjunto, analizan hasta qué punto los modelos de traducción automática son capaces de resolver tareas que requieren aprender a partir de muy pocos ejemplos, por lo que diferencia de una tarea de traducción automática convencional, estos problemas no proporcionan un corpus amplio, sino un conjunto mínimo de expresiones paralelas a partir del cual el sistema debe inferir léxico, reglas gramaticales y fenómenos morfológicos o fonológicos. Los autores evalúan aproximaciones de traducción automática previas a la aparición de los LLMs conversacionales actuales. En concreto, comparan métodos estadísticos simples con modelos neuronales de traducción de tipo secuencia a secuencia y arquitecturas profundas de traducción automática. Estos sistemas se entrenan o ajustan sobre los pocos pares disponibles en cada puzzle, de forma que el experimento mide si son capaces de generalizar desde un conjunto de entrenamiento extremadamente pequeño. Por este motivo, el trabajo resulta especialmente relevante como antecedente: no evalúa modelos como podrían ser GPT o Claude, sino técnicas que hoy pueden considerarse en gran parte obsoletas frente a los LLMs actuales.

Los resultados muestran que tanto los métodos estadísticos como los modelos neuronales evaluados tienen un rendimiento insuficiente en este tipo de problemas. De los experimentos realizados, se deduce que los modelos de traducción basados en patrones superficiales y grandes cantidades de datos no son adecuados para puzzles que requieren razonamiento metalingüístico, deducción y revisión de hipótesis a partir de unos pocos ejemplos. Esta conclusión es importante para este trabajo porque justifica el paso de la traducción automática convencional hacia estrategias de prompting más explícitas, donde el modelo debe construir un diccionario y reglas gramaticales antes de traducir.

2.3.2. modeLing

Chi y cols. (2024) presentan *modeLing*, un conjunto de problemas construido manualmente y diseñado específicamente para evaluar razonamiento lingüístico reduciendo el riesgo de contaminación de datos. A diferencia de otros benchmarks basados directamente en problemas ya publicados de la IOL, modeLing utiliza problemas creados para el propio trabajo, lo que disminuye la posibilidad de que los modelos hayan visto los enunciados durante el preentrenamiento.

El conjunto contiene 48 problemas y 272 preguntas, basados en 19 lenguas con presencia digital muy reducida. Los ejercicios cubren cuatro tipos principales de razonamiento, que son el orden entre nombre y adjetivo, orden de constituyentes, posesión y semántica. Además, los autores realizan una prueba sin ejemplos de referencia para detectar posible contaminación de datos. En la evaluación se prueban seis modelos de la familia GPT: Ada, Babbage, Curie, Davinci, GPT-3.5 y GPT-4. Los cuatro primeros pertenecen a la familia GPT-3; tomando como referencia los tamaños descritos por Brown y cols. (2020), Ada, Babbage, Curie y Davinci corresponden aproximadamente a modelos de 350M, 1.3B, 6.7B y 175B parámetros, respectivamente. En cambio, el tamaño exacto de GPT-3.5 y GPT-4 no es público, por lo que el trabajo los trata como modelos cerrados de mayor capacidad. Cabe mencionar que esta aclaración es importante porque varios de los modelos evaluados, así como Ada, Babbage y Curie, han quedado obsoletos frente a los modelos actuales. Los autores comparan cuatro configuraciones de prompting: un prompt mínimo, un prompt ajustado manualmente por una persona medallista de la Olimpiada Internacional de Lingüística, un *chain-of-thought* básico y un *full chain-of-thought* con un ejemplo de razonamiento paso a paso. Los resultados muestran una diferencia clara entre modelos pequeños y grandes. Ada, Babbage y Curie quedan prácticamente en cero en todas las configuraciones, con valores máximos de 0.011, 0.018 y 0.022 de exactitud, respectivamente. En cambio, Davinci alcanza hasta 0.514 con *full chain-of-thought*, GPT-3.5 obtiene valores alrededor de 0.40 y GPT-4 alcanza el mejor resultado, con 0.607 de exactitud en la configuración *full chain-of-thought*.

Estos resultados son relevantes para este trabajo por dos motivos. En primer lugar, muestran que los modelos grandes sí pueden resolver parte de los puzzles mediante razonamiento con pocos ejemplos, aunque el rendimiento sigue lejos de ser perfecto. En segundo lugar, indican que cambiar el tipo de prompt no siempre produce mejoras grandes: en *modeLing*, las diferencias entre prompt mínimo, prompt ajustado y razonamiento paso a paso son menores que las diferencias entre tamaños de modelo. Esto refuerza la necesidad de analizar conjuntamente el efecto de la metodología y el modelo utilizado.

2.3.3. Linguini

Sánchez y cols. (2024) presentan LINGUINI, el benchmark utilizado como dataset base de mi trabajo. Está formado por 894 preguntas agrupadas en 160 problemas extraídos del corpus de la IOL, y cubre 75 lenguas de baja densidad de recursos. Los problemas se organizan en tres categorías principales:

1. **Transducción de secuencias** (*sequence transduction*): el modelo debe transformar una secuencia en otra, por ejemplo traduciendo una frase o relacionando expresiones equivalentes.
2. **Relleno de huecos** (*fill-in-blanks*): el modelo debe completar formas lingüísticas aplicando reglas morfológicas o fonológicas deducidas a partir de los ejemplos.
3. **Transliteración numérica** (*number transliteration*): el modelo debe convertir expresiones numéricas entre su representación textual y su representación mediante cifras.

Los resultados publicados muestran que estos problemas siguen siendo difíciles para los modelos actuales. Ninguno de los modelos evaluados supera el 25% de exactitud. El mejor

modelo cerrado, Claude 3 Opus, alcanza un 24,05% de exactitud, mientras que el mejor modelo abierto, Llama 3 70B, obtiene tan solo un 8,84%. Como aclaración, el término de exactitud (Exact Match) se refiere al porcentaje de respuestas que coinciden exactamente con la referencia esperada, por ello una respuesta parcialmente correcta puede recibir una puntuación baja si no coincide de forma exacta con la solución del benchmark. Como métrica complementaria, los autores también reportan chrF, que permite medir similitudes parciales a nivel de caracteres. Los valores resultantes quedan por debajo del rendimiento humano reportado en competiciones de la IOL, donde estudiantes de secundaria o preuniversitarios suelen superar los 82 puntos en los mejores resultados anuales (Sánchez y cols., 2024; International Linguistics Olympiad, 2026). Esta comparación debe interpretarse teniendo en cuenta el perfil de los participantes, y es que la IOL está dirigida a estudiantes preuniversitarios, habitualmente de entre 15 y 20 años.

El experimento más relevante de Linguini es el de la prueba de la ablación sin contexto. Al eliminar las frases de entrenamiento del prompt, el rendimiento de los modelos cae de forma clara. Por ejemplo, Claude 3 Opus pasa de 24,05 a 1,23, GPT-4o de 14,65 a 1,45 y Llama 3 70B de 8,17 a 1,67. Aunque parezca obvio, esto es una demostración clara de que la información proporcionada en el contexto tiene un papel crucial en la resolución. Aun así, no es posible descartar por completo la influencia del conocimiento adquirido durante el preentrenamiento, porque los datos exactos usados para entrenar muchos de estos modelos no son públicos. Por tanto, una caída fuerte sin contexto es un indicio de baja contaminación, pero no una prueba absoluta de que el modelo no haya visto nunca información relacionada con la lengua o con el problema.

2.3.4. LingOly

Bean y cols. (2024) presentan *LingOly*, un benchmark de mayor escala que los conjuntos revisados anteriormente, especialmente en comparación con *PuzzLing Machines*, *modeLing* y *LINGUINI*, publicado en NeurIPS 2024. El conjunto cubre más de 90 lenguas, incluidas algunas lenguas extintas, y contiene 1.133 problemas organizados en seis categorías lingüísticas: fonología, morfología, sintaxis, semántica, sistemas numéricos y problemas compuestos. Además, los problemas se clasifican en cinco niveles de dificultad humana.

Esta organización permite analizar con más detalle qué tipos de fenómenos resultan más difíciles para los modelos. Por ejemplo, el benchmark incluye problemas relacionados con cambios fonológicos, formación de palabras, orden de constituyentes, concordancia, posesión, sistemas de numeración y combinaciones de varios fenómenos dentro de un mismo ejercicio. Los autores evalúan once modelos de lenguaje y comparan sus resultados con un baseline sin contexto. Los resultados muestran que los modelos tienen especiales dificultades en los problemas de mayor nivel. Incluso el mejor sistema evaluado alcanza únicamente un 38,7% de exactitud en los ejercicios más complejos, aunque mejora en 24,7 puntos respecto a la versión sin contexto.

El trabajo también observa una tendencia general: los modelos propietarios de mayor tamaño suelen superar a los modelos abiertos, y el rendimiento tiende a ser mayor en lenguas con más presencia digital.

2.3.5. Comparación entre los principales benchmarks

Los cuatro conjuntos revisados tienen objetivos complementarios. *PuzzLing Machines* plantea uno de los primeros marcos para usar problemas de tipo piedra Rosetta como prueba de aprendizaje con pocos datos. *modeLing* se centra en reducir el riesgo de contaminación mediante problemas creados expresamente para la evaluación. *Linguini* amplía la escala del análisis y proporciona el marco experimental utilizado en este trabajo. Por último, *LingOly* incorpora una mayor variedad de formatos, lenguas y niveles de dificultad.

La Tabla 2.1 resume las características principales de estos benchmarks. En el caso de *PuzzLing Machines*, el artículo describe el conjunto como cercano a 100 puzzles, mientras que el conteo del dataset utilizado corresponde a 96 problemas y 924 preguntas.

Tabla 2.1: Comparación de los principales benchmarks de olimpiadas lingüísticas para LLMs.

Benchmark	Año	Tamaño	Rasgo principal
<i>PuzzLing Machines</i>	2020	96 problemas, 924 preguntas	Antecedente centrado en problemas de tipo piedra Rosetta.
<i>modeLing</i>	2024	48 problemas, 272 preguntas	Problemas originales diseñados para reducir el riesgo de contaminación de los datos de entrenamiento.
<i>Linguini</i>	2024	160 problemas, 894 preguntas	Benchmark de referencia utilizado en los experimentos de este trabajo.
<i>LingOly</i>	2024	1.133 preguntas individuales	Conjunto de mayor tamaño, con distintos formatos y niveles de dificultad.

En conjunto, estos trabajos muestran que los problemas de olimpiadas lingüísticas son una herramienta útil para estudiar la capacidad de los modelos de lenguaje para generalizar a partir de pocos ejemplos. También dejan claro que el rendimiento sigue siendo limitado y que la forma de presentar la información al modelo puede influir de manera importante en los resultados. Esta última idea es especialmente relevante para este trabajo, ya que la aportación principal no consiste en proponer un nuevo benchmark, sino en comparar distintas formas de guiar el razonamiento del modelo sobre los mismos problemas.

2.4. Estrategias de prompting para razonamiento lingüístico

Ante los resultados todavía limitados del prompting directo en tareas de razonamiento lingüístico, varios trabajos han explorado estrategias más elaboradas para guiar al modelo en su razonamiento y en sus conclusiones. En los benchmarks revisados anteriormente, estas mejoras se han planteado principalmente mediante cambios en la forma de presentar el problema: añadir ejemplos de razonamiento, organizar los pasos de análisis, introducir instrucciones más explícitas o pedir al modelo que revise sus hipótesis antes de responder (Chi y cols., 2024;

Zhu y cols., 2025; Madaan y cols., 2023). Esta sección no se limita únicamente a trabajos sobre olimpiadas lingüísticas, sino que recoge también estrategias generales de prompting que resultan relevantes para este tipo de tareas. En todos los casos, se trata de métodos que no modifican los parámetros del modelo, ya que el objetivo es evaluar qué puede hacer un modelo ya entrenado a partir de la información incluida en el prompt, sin recurrir a fine-tuning ni a entrenamiento adicional. Esta decisión es especialmente importante en lenguas de baja densidad de recursos, donde no suele haber datos suficientes para ajustar un modelo de forma fiable.

2.4.1. Prompting analógico

Ramji y Ramji (2024) proponen una estrategia basada en el razonamiento analógico. En lugar de depender únicamente de los ejemplos incluidos en el problema, el modelo genera o recupera ejemplos auxiliares relacionados con fenómenos lingüísticos similares. Dicho de otra manera, el modelo trata de aprovechar el conocimiento general adquirido durante el preentrenamiento. Por ejemplo, si una lengua desconocida presenta una estructura sintáctica poco habitual, el modelo puede intentar relacionarlo con estructuras semejantes presentes en otras lenguas que conoce mejor.

Esta estrategia puede facilitar la identificación de determinados patrones, pero también introduce una limitación importante, además de desviarse de nuestro objetivo principal, y es que el razonamiento deja de depender exclusivamente de la información proporcionada en el enunciado. Otro problema es que el modelo, al igual que saca conclusiones verídicas, también puede apoyarse en analogías incorrectas o trasladar reglas de una lengua a otra sin suficiente evidencia.

En mi investigación no se utiliza prompting analógico, ya que uno de los objetivos es evaluar hasta qué punto el modelo puede resolver los problemas a partir de los ejemplos disponibles, sin recibir pistas sobre la lengua ni sobre su familia lingüística. No obstante, esta aproximación podría resultar interesante como posible línea de ampliación futura.

2.4.2. Gramática en contexto

Tanzer y cols. (2024) estudian otra aproximación: proporcionar al modelo una gramática descriptiva de la lengua como parte del contexto. Este enfoque, denominado *one-book prompting*, permite trabajar con lenguas para las que existe documentación lingüística, aunque no se disponga de grandes corpus textuales. A diferencia de los problemas de olimpiadas lingüísticas, el modelo no debe deducir todas las reglas exclusivamente a partir de ejemplos. En su lugar, puede consultar una descripción explícita de la lengua y utilizarla para traducir nuevas frases.

Los resultados muestran que incluir una gramática puede mejorar el rendimiento del modelo, especialmente cuando se combina con ejemplos. Resulta que muchas lenguas con pocos recursos digitales sí disponen de gramáticas, diccionarios o estudios elaborados por especialistas, por lo que esta estrategia parece prometedora para aplicaciones reales de traducción y documentación lingüística. Sin embargo, este enfoque responde a un objetivo diferente del que se pretende abordar. Aquí el objetivo está en evaluar la capacidad de deducción a partir de un conjunto reducido de pares de traducción, sin incorporar documentación externa sobre la lengua.

Sin embargo, este enfoque también ha recibido críticas. Aycock y cols. (2025) señalan que la mejora observada al añadir una gramática no siempre se debe a que el modelo esté usando realmente las explicaciones gramaticales. En muchos casos, la mejora parece venir sobre todo de los ejemplos de traducción que acompañan a la gramática. Es decir, el modelo aprovecha mejor los pares de frases ya traducidas que las reglas lingüísticas descritas en el documento. Los autores indican que una gramática puede ser útil para tareas como decidir si una frase está bien formada o predecir glosas, pero no encuentran pruebas claras de que los LLMs sepan usar una gramática completa para traducir mejor. Esta crítica es importante para este trabajo porque obliga a distinguir bien qué tipo de ayuda recibe el modelo: ejemplos paralelos, reglas explícitas o instrucciones para razonar paso a paso.

2.4.3. Aprendizaje en contexto con ejemplos

El aprendizaje en contexto (*in-context learning*, ICL) es la capacidad de un modelo de lenguaje para realizar una tarea nueva a partir de la información incluida directamente en el prompt, sin necesidad de actualizar sus parámetros. Esta capacidad se acerca más a lo que pretendemos, ya que en los problemas de olimpiadas lingüísticas, la propiedad *in-context* resulta fundamental. El modelo recibe un conjunto reducido de pares formados por una expresión en una lengua desconocida y su traducción. A partir de estos ejemplos debe deducir vocabulario, identificar regularidades morfológicas y aplicar las reglas extraídas a nuevas frases.

Es importante diferenciar este mecanismo del *few-shot prompting*. En una configuración *few-shot*, el modelo recibe ejemplos adicionales que muestran cómo debe resolverse la tarea o cómo debe estructurarse la respuesta. Estos ejemplos no forman parte necesariamente del problema lingüístico concreto, sino que actúan como demostraciones previas. Siempre que se trate de dar información adicional como apoyo al modelo, desviamos el objetivo principal de comprobar la capacidad de razonamiento metalingüístico de los modelos de lengua.

Sánchez y cols. (2024) comparan distintas configuraciones con un número variable de demostraciones y observan resultados extraídos de la respuesta de los modelos. Algunos de ellos mejoran al recibir ejemplos adicionales, mientras que otros empeoran. La explicación más plausible hasta el momento es que las demostraciones introducen información útil sobre el formato de la tarea, pero también pueden añadir ruido cuando proceden de lenguas o estructuras poco relacionadas con el problema que se desea resolver. Esta observación es bastante interesante para la presente investigación: proporcionar más información no garantiza por sí solo una mejora del rendimiento. Es crucial estudiar cómo se presenta esa información, en qué orden se incorpora y si el modelo es capaz de mantener hipótesis coherentes a lo largo de la resolución.

2.4.4. Razonamiento paso a paso

Zhu y cols. (2025) proponen una estrategia de descomposición paso a paso inspirada en el *chain-of-thought prompting* (Wei y cols., 2022). La idea general consiste en dividir un problema complejo en pasos intermedios más sencillos de manejar, en lugar de pedir directamente la respuesta final. En el contexto de las olimpiadas lingüísticas, este razonamiento puede consistir en identificar primero el vocabulario básico, después analizar los patrones morfológicos, formular reglas gramaticales y, finalmente, aplicar esas reglas a las frases de prueba.

En el estudio de la descomposición progresiva, en lugar de presentar todos los ejemplos de entrenamiento de golpe, los autores organizan la información en etapas lingüísticas (concretamente, en 4 pasos): primero se presenta el léxico y el orden de palabras básico, después los patrones fonológicos, luego la morfosintaxis y finalmente la sintaxis compleja. En cada etapa, el modelo actualiza su comprensión de la lengua antes de pasar a la siguiente. Este enfoque obtiene mejoras significativas respecto al prompting directo en lenguas con morfología verbal compleja. Aquí comprobaremos dicha estrategia, ya que resulta especialmente observar que la forma de ordenar y estructurar los ejemplos puede influir en el rendimiento del modelo, incluso cuando la información lingüística disponible es la misma.

2.4.5. Autocorrección y revisión de respuestas

Otra línea de trabajo relacionada es la autocorrección iterativa. El paradigma *Self-Refine*, propuesto por Madaan y cols. (2023), plantea que un modelo puede generar una primera respuesta, producir una crítica sobre ella y después elaborar una versión revisada a partir de esa crítica. Aplicado a tareas de razonamiento lingüístico, este planteamiento permite estudiar si el modelo es capaz de detectar contradicciones internas en su propia respuesta. La idea es que si se formula una regla gramatical durante el análisis, pero después genera una traducción que no respeta esa regla, una fase de revisión debería permitir identificar el problema y corregirlo.

Este enfoque puede tener una limitación importante, que es la retroalimentación procede del propio modelo. Si la primera interpretación contiene una regla equivocada, el modelo puede utilizar esa misma regla como criterio de revisión y reforzar el error en lugar de corregirlo, aunque se le indique explícitamente que esta primera hipótesis puede contener errores. Por tanto, la autocorrección puede ser útil en algunos casos, pero no garantiza necesariamente una mejora del resultado final.

La estrategia de la autocorrección también se incorpora en esta investigación, se diseña prompt que pida una segunda fase de revisión, en la que el modelo debe comprobar la coherencia entre sus traducciones, su diccionario y las reglas que ha formulado. Como veremos más adelante, los resultados no siempre mejoran respecto al baseline, lo que permite analizar también las limitaciones de este tipo de revisión.

2.5. Análisis de las dificultades de los LLMs en puzzles lingüísticos

Además de medir cuántas respuestas aciertan los modelos, varios trabajos recientes han intentado analizar por qué fallan en los problemas de olimpiadas lingüísticas. Esta cuestión es importante porque dos modelos pueden obtener una puntuación parecida y, sin embargo, cometer errores de base muy diferentes. Mientras que uno puede fallar por no identificar vocabulario básico, otro por no mantener una regla morfológica de forma consistente y otro por producir una traducción gramaticalmente plausible pero incompatible con los ejemplos del enunciado.

Choudhary y cols. (2025) realizan uno de los análisis más completos en esta dirección. En su trabajo analizan el rendimiento de modelos de lenguaje sobre 629 problemas en 41 lenguas de baja densidad de recursos, anotando cada puzzle con 50 características lingüísticas. Entre los modelos evaluados se incluyen sistemas cerrados y abiertos, como GPT-4o, Claude 3.5

Sonnet, Llama 3.1 8B, Llama 3.1 70B, Gemma 2 9B, Gemma 2 27B, Mistral 7B, Mixtral 8x7B y Qwen 2.5 7B. Las conclusiones que sacan resultan muy reveladoras y apuntan a dos factores principales. Por un lado, deducen que la complejidad morfológica parece ser uno de los elementos que más dificulta la resolución, y es que cuanto mayor es el número de morfemas por palabra, peor tienden a comportarse los modelos. Por otro, los modelos obtienen mejores resultados cuando las características del puzzle se parecen a patrones presentes en inglés, en español o en otras lenguas con alta representación digital. Además, los autores muestran que separar previamente las palabras en morfemas mejora el rendimiento, lo que sugiere que parte del problema está relacionado con la forma en que los modelos tokenizan y representan lenguas poco familiares (Choudhary y cols., 2025). Con ello, cabe destacar que no todos los puzzles tienen la misma dificultad, aunque tengan un número parecido de ejemplos para el entrenamiento. Una lengua con marcas de persona, número, caso o tiempo muy fusionadas en la palabra puede ser más difícil que otra con una segmentación más transparente. Por ejemplo, es más sencillo identificar el plural si las palabras que lo indican siempre contienen la terminación ‘s’, o el género si lo que cambia es el artículo anterior a la misma palabra. Por tanto, las diferencias de rendimiento entre lenguas no deben explicarse únicamente por el modelo o por la estrategia de prompting utilizada, sino también por las propiedades lingüísticas del propio puzzle, como también comprobaremos en los resultados de la investigación.

Un trabajo posterior de Majmudar y Filatova (2025) amplía la evaluación a modelos de razonamiento y estudia tanto la resolución como la generación de puzzles lingüísticos. El trabajo utiliza problemas de olimpiadas lingüísticas dirigidos a estudiantes de secundaria y evalúa modelos recientes, incluyendo o1 de OpenAI, junto con otros LLMs de alto rendimiento. Los resultados muestran que los modelos de razonamiento mejoran respecto a sistemas anteriores en varias categorías de puzzles, e incluso pueden superar a participantes humanos en algunos tipos de ejercicios. Sin embargo, los autores señalan que siguen apareciendo dificultades claras en problemas relacionados con sistemas de escritura poco familiares y con lenguas poco estudiadas o tipológicamente alejadas de las lenguas más presentes en los datos de entrenamiento (Majmudar y Filatova, 2025).

Este resultado sugiere que aumentar la capacidad de razonamiento del modelo puede mejorar el rendimiento, pero no elimina por completo las limitaciones asociadas a la distancia tipológica, la morfología compleja o la falta de familiaridad con determinados sistemas lingüísticos. Además, el propio trabajo muestra que resolver puzzles y generarlos no son tareas equivalentes: los modelos pueden acertar algunos problemas, pero tienen más dificultades para crear puzzles nuevos que sean correctos, coherentes y resolubles por una persona.

En conjunto, estos trabajos muestran que los errores de los modelos no son aleatorios, sino que suelen concentrarse en fenómenos concretos: marcas morfológicas pequeñas, distinciones de número o persona, alternancias internas de la palabra, sistemas de escritura no familiares o reglas que deben aplicarse de forma consistente a varias frases. Esta observación explica que, además de calcular métricas automáticas, sea necesario revisar cualitativamente algunos casos para entender qué tipo de razonamiento está realizando el modelo y dónde se rompe.

2.6. Síntesis y posicionamiento del trabajo

La revisión anterior permite situar el presente trabajo de fin de grado dentro de una línea de investigación reciente: la evaluación del razonamiento lingüístico de los modelos de lenguaje a partir de problemas de olimpiadas lingüísticas. Los trabajos revisados muestran que los modelos actuales pueden identificar algunos patrones a partir de pocos ejemplos, pero también que su rendimiento sigue estando lejos de ser estable. La dificultad depende de muchos factores: la lengua del puzzle, la complejidad morfológica, la dirección de traducción, el formato del prompt y la capacidad del modelo para mantener hipótesis coherentes.

A partir de este contexto, aquí no se propone un nuevo benchmark, sino que utiliza Linguini como base experimental para comparar distintas formas de presentar la información al modelo. El objetivo principal no es solo medir cuántas traducciones son correctas, sino observar si una estrategia de prompting más estructurada ayuda al modelo a construir un diccionario, formular reglas gramaticales y revisar sus propias hipótesis durante la resolución.

La metodología de este trabajo se parece a los estudios previos en que también evalúa modelos de lenguaje mediante problemas lingüísticos autocontenidos y utiliza ejemplos dentro del prompt para inducir reglas, como ocurre en benchmarks como *PuzzLing Machines*, *modeLing*, LINGUINI o *LingOly*. También comparte con trabajos sobre chain-of-thought, autocorrección y razonamiento guiado la idea de que no basta con pedir una respuesta final, sino que puede ser útil hacer explícitos pasos intermedios del análisis. Sin embargo, se diferencia de esos trabajos en que no propone nuevos problemas ni se centra únicamente en comparar modelos, sino en comparar varias metodologías de presentación y revisión sobre el mismo conjunto de puzzles. En particular, se estudian estrategias como la presentación incremental acumulativa, la inspiración en una resolución humana y la autocorrección iterativa, manteniendo constante el dataset para observar mejor el efecto del diseño del prompt.

La línea de trabajo general consiste en establecer varias metodologías, diferentes entre sí, y observar como se comportan con puzzles de distintas familias y estructuras. Con estas estrategias, nos situaremos entre la evaluación cuantitativa y el análisis cualitativo. Por un lado, se comparan modelos y metodologías mediante métricas automáticas y, por otro, se revisan casos concretos para entender qué hipótesis formula el modelo, qué errores aparecen y cómo evoluciona su razonamiento a lo largo del proceso.

Cabe destacar que la finalidad no es demostrar que los modelos razonan de la misma manera que una persona, sino analizar si una organización más guiada del proceso de resolución puede producir respuestas más correctas, coherentes y revisables en problemas de olimpiadas lingüísticas.

3. Objetivos

Este capítulo presenta el objetivo principal del trabajo, los objetivos específicos que se derivan de él y las principales restricciones que han condicionado su desarrollo.

3.1. Objetivo principal

El objetivo principal de esta investigación es diseñar, implementar y evaluar distintas estrategias de prompting para estudiar hasta qué punto los modelos de lenguaje de gran escala pueden resolver problemas de olimpiadas lingüísticas a partir de un conjunto reducido de ejemplos, sin recibir información explícita previa sobre la lengua analizada y apoyándose únicamente en el razonamiento metalingüístico inducido por el contexto. Algunas estrategias se inspiran en trabajos previos, como el baseline de aprendizaje en contexto, el razonamiento paso a paso y la autocorrección iterativa; otras se plantean como aportaciones propias o adaptaciones específicas para este trabajo, como la presentación incremental acumulativa, su variante combinatoria, y la estrategia basada en razonamiento humano. Veremos cómo se comporta el modelo y la forma en la que llega a sus conclusiones cuando se le presentan diferentes formatos del problema.

3.2. Objetivos específicos

Basándonos en lo que queremos conseguir, del objetivo principal derivan los siguientes objetivos específicos que ayudarán a desarrollar el trabajo paso a paso:

1. **Revisar el estado de la cuestión** sobre razonamiento lingüístico con modelos de lenguaje de gran escala, prestando especial atención a los benchmarks derivados de problemas de olimpiadas lingüísticas, como Linguini, LingOly y modeLing, así como a las estrategias de prompting más relevantes de la literatura reciente. Sobre todo, destacar el contenido existente a partir de los trabajos previos en la materia para ver qué se podría aportar con este proyecto.
2. **Seleccionar un conjunto de puzzles de traducción** procedentes del benchmark Linguini, priorizando problemas de lenguas de baja densidad de recursos, con suficiente número de ejemplos de entrenamiento y test, y pertenecientes a distintas familias lingüísticas. Aunque los resultados que podamos obtener sean escasos debido a las limitaciones de peticiones a modelos abiertos en la nube, escogeremos aquellos que sean relevantes para responder a las preguntas planteadas.
3. **Diseñar e implementar varias estrategias de prompting de metodologías ya estudiadas e incorporar nuevas, comparables entre sí**, incluyendo una estrategia base de aprendizaje in-context, una estrategia de presentación incremental acumulativa,

una estrategia paso a paso organizada por niveles lingüísticos, una estrategia inspirada en razonamiento humano y una estrategia de autocorrección iterativa. Todas las estrategias mencionadas se describirán con más detalle en el Capítulo 4. Además, para cada problema se realiza una prueba adicional sin ejemplos de entrenamiento, en la que se pide al modelo que responda sin recibir los pares de traducción del enunciado. Esta prueba sirve como control para comprobar si el modelo depende realmente del contexto proporcionado o si podría estar respondiendo a partir de conocimiento previo adquirido durante el preentrenamiento.

4. **Plantear una extensión combinatoria de la estrategia incremental, propia del trabajo**, orientada a evaluar en trabajos futuros todas las combinaciones posibles de ejemplos de entrenamiento para cada tamaño de subconjunto. Esta ampliación permitiría analizar si el rendimiento depende del orden y de la selección concreta de los ejemplos proporcionados al modelo. Debido a la cantidad de peticiones que requiere dicha estrategia, esta se probará con un número escaso de problemas, por lo que aún quedaría como una ampliación futura a investigar.
5. **Construir un sistema experimental reproducible** que automatice la carga de puzzles, la generación de prompts, la ejecución de los modelos, la extracción de traducciones, el cálculo de métricas y el almacenamiento de logs completos para su posterior revisión.
6. **Evaluar las estrategias implementadas** sobre puzzles reales del benchmark Linguini, utilizando modelos de lenguaje de distintas familias y tamaños, y centrando el análisis principal en la dirección de traducción desde la lengua desconocida hacia el inglés. Como ampliación, se probará el cambio de dirección de traducción, partiendo de base de que los resultados de esta puedan ser poco prometedores.
7. **Comparar los resultados obtenidos** mediante métricas automáticas de traducción, principalmente BLEU (Papineni y cols., 2002), chrF++ (Popović, 2017), TER (Snover y cols., 2006) y Exact Match (Sánchez y cols., 2024), con el fin de identificar en qué condiciones cada estrategia mejora o empeora respecto al baseline directo.
8. **Realizar un análisis cualitativo** de casos representativos, comparando el proceso de resolución humana con el razonamiento explícito generado por los modelos, para identificar patrones de error, contradicciones internas y diferencias en la forma de construir hipótesis lingüísticas. Esto además servirá para probar una de las metodologías incorporadas: el entrenamiento basado en el razonamiento humano.
9. **Publicar el código desarrollado en un repositorio accesible**, con el fin de facilitar la reutilización y reproducibilidad del trabajo. El repositorio incluirá el código de evaluación y las instrucciones necesarias para repetir los experimentos. La URL del repositorio se recoge en el anexo A.

3.3. Restricciones y condicionantes

El desarrollo del trabajo ha tenido una serie de complicaciones durante su realización, debido a las siguientes restricciones:

Acceso a los modelos: se utilizan modelos accesibles mediante APIs externas gratuitas, principalmente Groq¹ y OpenRouter². Durante el desarrollo también se probaron modelos de Cerebras³ y SambaNova⁴. Esta decisión permite trabajar con modelos de distintos tamaños sin disponer de infraestructura propia, pero limita la selección de modelos disponibles y hace que el sistema dependa de servicios externos. Además, la disponibilidad de estos modelos abiertos ha ido cambiando durante el desarrollo, lo que atrasó la continuidad y redujo la cantidad posible de pruebas a realizar.

Límites de tasa: las APIs gratuitas imponen límites de peticiones por minuto y por día. Esto condiciona el número de experimentos que pueden ejecutarse en una misma sesión y obliga a introducir pausas entre llamadas. La restricción es especialmente importante en las estrategias iterativas, porque generan varias peticiones para un mismo puzzle. Este es el principal motivo de la escasez de las pruebas de la metodología iterativa combinatoria, la cual era de especial importancia debido a ser una aportación propia a la investigación.

Coste combinatorio de la metodología iterativa combinatoria acumulativa (M2.2): la extensión combinatoria de la estrategia de prompt incremental aumenta de forma considerable el número de llamadas necesarias. Para un puzzle con n pares de entrenamiento, el número de subconjuntos evaluados sería:

$$\sum_{k=2}^n \binom{n}{k}$$

Por ejemplo, para $n = 10$ serían necesarias 1013 iteraciones por puzzle y modelo. Por este motivo, M2.2 se plantea como una ampliación metodológica, pero su evaluación completa queda fuera del alcance principal de esta entrega. Sin embargo, tal y como veremos en el apartado de Resultados, ha sido posible lanzar par de pruebas pese a su coste elevado.

Datos de preentrenamiento desconocidos: no es posible conocer con precisión todos los textos utilizados durante el entrenamiento de los modelos evaluados. Por este motivo, no puede garantizarse que las lenguas analizadas sean completamente desconocidas para el modelo o que no haya visto información relacionada con ellas. Para controlar este problema de forma más estricta, algunos trabajos utilizan la encriptación de los caracteres, evitando así que el modelo pueda apoyarse directamente en conocimiento previo. Aun así, aplicar este tipo de control requeriría diseñar e implementar un proceso adicional de transformación y validación de los datos, por lo que queda fuera del alcance de este trabajo. En este caso, el control de verificación (M0) se utiliza únicamente para detectar posibles indicios de conocimiento previo, no para demostrar la ausencia total de contaminación en los datos de entrenamiento.

¹<https://groq.com/>

²<https://openrouter.ai/>

³<https://www.cerebras.ai/inference>

⁴<https://cloud.sambanova.ai/>

Alcance experimental: aunque el sistema permite trabajar con puzzles en ambas direcciones de traducción, el análisis principal de resultados se centra en la dirección lengua desconocida → inglés. La investigación más detallada sobre la dirección inversa, inglés → lengua desconocida, se reserva como línea de ampliación futura debido al aumento de complejidad experimental y al tiempo disponible.

4. Metodología y datos

Este capítulo describe cómo se ha planteado la parte experimental del trabajo. En primer lugar, se presenta el conjunto de datos utilizado y los criterios seguidos para seleccionar los puzzles. A continuación, se describen los modelos evaluados, las estrategias de prompting, las métricas empleadas y el análisis cualitativo realizado sobre un caso representativo.

4.1. Dataset: Linguini

Los experimentos se realizan sobre puzzles extraídos del benchmark Linguini, construido a partir del corpus de la Olimpiada Internacional de Lingüística (IOL). Dentro de este benchmark se seleccionaron principalmente problemas de tipo *sequence transduction*, ya que son los que mejor se ajustan al objetivo de este trabajo, que consiste en evaluar si un modelo puede deducir correspondencias entre una lengua poco conocida y el inglés a partir de un conjunto reducido de ejemplos. Esta categoría incluye tareas de traducción y permite comparar los resultados con trabajos previos que también utilizan Linguini como referencia (Sánchez y cols., 2024; Zhu y cols., 2025).

4.1.1. Criterios de selección

La selección de puzzles no se realizó de forma aleatoria, sino siguiendo varios criterios prácticos y lingüísticos. En primer lugar, se priorizaron lenguas pertenecientes a familias distintas, con el fin de aumentar la diversidad tipológica del conjunto. Como se ha señalado en la sección 2.5, el rendimiento de los modelos puede variar según las propiedades lingüísticas del puzzle y el tipo de problema que se le presenta. En segundo lugar, se seleccionaron los puzzles que tuvieran al menos ocho pares de entrenamiento, ya que las estrategias incrementales necesitan varios pasos para poder construir y revisar hipótesis. En tercer lugar, se mantuvieron únicamente los puzzles con al menos tres frases de test, para evitar que las métricas dependieran de una sola respuesta. Por último, se descartaron o revisaron los casos en los que el control exploratorio (verificación de contaminación de datos) indicaba posibles señales de conocimiento previo por parte del modelo (véase la sección 4.3.2).

El sistema experimental permite trabajar con puzzles en dos direcciones: de la lengua desconocida al inglés (*lang2en*) y del inglés a la lengua desconocida (*en2lang*). Esta segunda dirección es especialmente interesante porque obliga al modelo a producir formas en una lengua que no conoce, no solo a interpretar su significado. No obstante, por razones de alcance y tiempo, el análisis principal de resultados se centra en la dirección *lang2en*. Los puzzles en dirección inversa se conservaron durante la preparación del conjunto de datos y quedan como base para una ampliación del trabajo.

4.1.2. Puzles seleccionados

El conjunto final comprende 11 puzles de traducción en dirección lengua desconocida $\rightarrow$ inglés, correspondientes a 11 lenguas y 10 familias lingüísticas distintas. Esta selección cubre regiones geográficas diversas, incluyendo Asia, Oceanía, América, África y América del Norte.

La Tabla 4.1 recoge las características principales de cada puzle. La columna ID corresponde al identificador original del problema dentro del dataset LINGUINI. Las columnas Train y Test indican, respectivamente, el número de frases de entrenamiento y de prueba utilizadas en cada puzle. Cabe mencionar que en la mayoría de los casos se utiliza el nombre de la lengua en español, y en el caso de que no exista o no se conozca una forma española asentada, se conserva el nombre original o el nombre empleado en el dataset.

Tabla 4.1: Puzles utilizados en los experimentos. Benchmark LINGUINI.

ID	Lengua	Familia	Región	Train	Test
012018020100	Hankun	Sino-Tibetana	India	10	6
012022030100	Ngemba	Níger-Congo	Camerún	16	6
012019010100	Yonggom	Trans-Nueva Guinea	Papúa-NG	11	6
012015040100	Warlpiri	Pama-Ñungana	Australia	17	6
012023020100	Apurinã	Arawak	Brasil	16	3
012023030100	Marind costero	Anim	Indonesia	17	5
012012010200	Dyirbal	Pama-Ñungana	Australia	11	3
012016030100	Nubio kunuz	Nilo-Sahariana	Egipto/Sudán	10	5
012008050100	Inuktitut	Eskaleut	Canadá	12	6
012005010100	Tzeltal	Maya	México	15	4
012006010100	Lakota	Siouan	EE. UU.	13	4

4.1.3. Formato de los datos

Los puzles de Linguini se distribuyen originalmente en formato JSONL (*JSON Lines*), un formato en el que cada línea del archivo contiene un objeto JSON independiente. Para facilitar la ejecución de los experimentos, se implementó un conversor que transforma cada problema seleccionado en un archivo JSON independiente con una estructura homogénea. Cada archivo contiene los campos `id`, `language`, `language_family`, `translation_direction`, `train_pairs` y `test_pairs`. Esta conversión permite separar claramente los ejemplos que se incluyen en el prompt de las frases reservadas para evaluación, y facilita añadir nuevos puzles sin modificar el resto del sistema.

4.2. Modelos evaluados

Se evaluaron tres modelos de lenguaje de gran escala accesibles mediante APIs externas gratuitas. La elección de estos modelos responde a dos criterios: disponer de sistemas de distinto tamaño y mantener un coste computacional asumible durante el desarrollo del trabajo. La Tabla 4.2 resume sus características principales.

Los dos primeros modelos pertenecen a la familia Llama y permiten comparar el efecto del tamaño manteniendo una familia de modelos similar. El modelo GPT-OSS-120B se incorpora como modelo de mayor tamaño y de una familia diferente, además de ser un modelo de razonamiento, con el objetivo de comprobar si los patrones observados se mantienen más allá de los modelos Llama, que son modelos de lenguaje, no entrenados explícitamente para el razonamiento. No se proporcionan ejemplos adicionales de resolución fuera de los incluidos en cada estrategia, salvo en aquellas metodologías en las que esta información forma parte explícita del diseño experimental.

Tabla 4.2: Modelos evaluados.

Denominación	Modelo	Organización	Parámetros	Nivel	API
Llama-8B	Llama 3.1 8B Instant	Meta	8B	Débil	Groq
Llama-70B	Llama 3.3 70B Versatile	Meta	70B	Medio	Groq
GPT-OSS-120B	GPT-OSS 120B	OpenAI	120B	Fuerte	OpenRouter

4.3. Estrategias de prompting

Se diseñó un control exploratorio previo, que vamos a identificar como M0, y cinco estrategias principales de traducción, identificadas como M1 a M5. Todas las estrategias de traducción comparten una misma instrucción metalingüística base, pero se diferencian en la forma de presentar los ejemplos al modelo y en el tipo de razonamiento que se le solicita.

4.3.1. Instrucción metalingüística base

Las estrategias M1 a M5 parten de una instrucción común que define el papel del modelo durante la resolución. Se le pide que actúe como un analista lingüístico que nunca ha visto la lengua del puzzle, que razone únicamente a partir de los ejemplos proporcionados y que trate sus hipótesis como provisionales. También se le indica que no utilice conocimiento previo sobre la lengua ni sobre su familia lingüística, y que construya explícitamente un diccionario y un conjunto de reglas gramaticales antes de producir las traducciones finales.

Esta instrucción busca forzar un comportamiento más cercano al concepto de razonamiento metalingüístico: observar ejemplos, identificar patrones, proponer reglas y aplicarlas de forma sistemática. El planteamiento se inspira en el prompting metalingüístico de Zhu y cols. (2025) y en los principios del *chain-of-thought* (Wei y cols., 2022), aunque en este trabajo se adapta al formato concreto de los puzzles de Linguini.

4.3.2. M0: Control exploratorio previo

Antes de ejecutar las estrategias de traducción, se aplica un control exploratorio previo para detectar posibles indicios de conocimiento de la lengua por parte del modelo. En esta prueba, el modelo recibe una única frase del conjunto de test sin ningún ejemplo de entrenamiento y se le pide que intente traducirla. El objetivo no es evaluar todavía la capacidad de aprendizaje in-context, sino comprobar si el modelo parece conocer la respuesta antes de

recibir los pares de traducción del puzle. Para dicha comprobación se utiliza chrF++, que es una métrica automática que compara la respuesta generada con la referencia esperada a partir de coincidencias de caracteres y palabras. Esta métrica resulta adecuada en este caso porque permite detectar similitudes parciales entre dos frases, incluso cuando no coinciden exactamente. Por ejemplo, si la traducción esperada fuera *the man sees the dog* y el modelo respondiera *a man sees a dog*, la respuesta no sería una coincidencia exacta, pero sí compartiría buena parte de sus caracteres y palabras. Por este motivo, chrF++ puede servir como una señal aproximada de cercanía entre la traducción generada y la solución esperada.

Por ello, si la traducción obtenida alcanza una puntuación chrF++ igual o superior a 20, el caso se marca para revisión manual, ya que podría indicar que el modelo posee algún conocimiento de la lengua (como claramente es el caso del ejemplo mencionado anteriormente) y, por lo tanto, podría existir contaminación de datos. No interpretaremos ese umbral como una prueba definitiva, sino como una señal de alerta. Es importante mencionar que en muchos casos, una traducción generada al azar puede compartir algunas palabras o caracteres con la referencia, especialmente en frases cortas o con palabras frecuentes como artículos, pronombres o nombres propios. Por este motivo, una puntuación inferior a 20 se considera compatible con una respuesta poco informada, mientras que una puntuación igual o superior merece ser revisada manualmente antes de incluir el puzle en los experimentos principales.

El funcionamiento general de este control exploratorio se resume en la Figura 4.1.

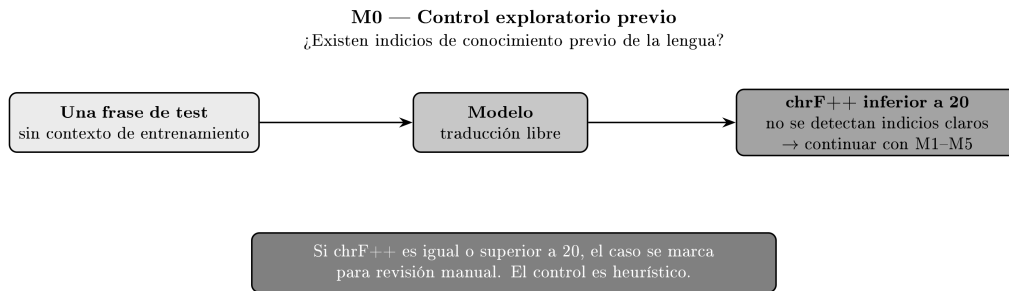


Figura 4.1: Funcionamiento del control exploratorio previo M0.

4.3.3. M1: Baseline

M1 es la estrategia de referencia. En ella, el modelo recibe todos los pares de entrenamiento del puzle en un único prompt y debe producir directamente tres elementos: un diccionario, un conjunto de reglas gramaticales y las traducciones de las frases de test.

Esta estrategia permite medir el rendimiento del modelo en una configuración sencilla, equivalente al prompting directo utilizado como punto de partida en trabajos previos como Sánchez y cols. (2024). Su función dentro del experimento es servir como baseline frente al que comparar las demás metodologías. El funcionamiento general de esta estrategia se resume en la Figura 4.2.

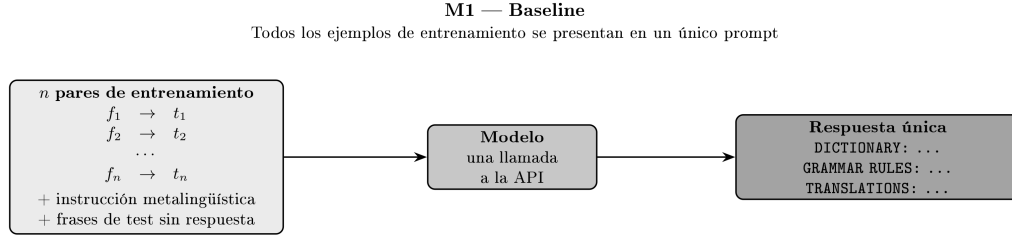


Figura 4.2: Funcionamiento de la estrategia baseline M1.

Los prompts completos utilizados para esta estrategia, así como para el resto de metodologías evaluadas, se presentan en el Capítulo 5.

4.3.4. M2: Presentación incremental acumulativa

M2 es la estrategia incremental del trabajo. A diferencia de M1, no presenta todos los ejemplos de entrenamiento desde el principio. En su lugar, los ejemplos se incorporan progresivamente mediante prefijos de tamaño creciente: primero las dos primeras frases, después las tres primeras, y así sucesivamente hasta llegar al conjunto completo de entrenamiento. Para evitar ejecuciones excesivamente largas en puzzles con muchos ejemplos, además de la limitación de tokens diarios, el número máximo de pasos se limita a $\min(n - 1, 15)$.

En cada paso, el modelo recibe los nuevos ejemplos junto con la hipótesis acumulada del paso anterior, formada por el diccionario y las reglas gramaticales ya inferidas. A partir de esa información debe revisar su análisis y producir una versión actualizada de sus hipótesis.

El prompt de M2 obliga al modelo a seguir cuatro fases:

1. **Análisis** (STEP A): razonar sobre los nuevos ejemplos antes de actualizar ninguna hipótesis, identificando posibles morfemas, correspondencias léxicas y contradicciones con reglas anteriores.
2. **Diccionario revisado** (STEP B): añadir nuevas entradas, corregir entradas imprecisas y eliminar aquellas que hayan quedado refutadas por los nuevos ejemplos. Las eliminaciones se marcan explícitamente con el formato `[DELETED: razón]`.
3. **Reglas revisadas** (STEP C): actualizar las reglas gramaticales con la misma lógica, manteniendo solo aquellas que sean compatibles con la evidencia disponible.
4. **Traducciones** (STEP D): traducir las frases de test utilizando el diccionario y las reglas actualizadas.

La parte más importante de M2 no es solo que el modelo reciba los ejemplos poco a poco, sino que se le obliga a revisar lo que ya había formulado. Esta decisión se toma porque, durante las pruebas iniciales, los modelos tendían a mantener reglas incorrectas incluso cuando aparecían ejemplos que las contradecían. Por ello, el prompt no plantea el conocimiento acumulado como definitivo, sino como una hipótesis revisable.

Para la comparación principal entre metodologías se utiliza el último paso de M2, es decir, aquel en el que el modelo ya ha recibido todos los ejemplos de entrenamiento. El mejor paso

observado según chrF++ se conserva únicamente como información diagnóstica, útil para estudiar si el rendimiento mejora de forma progresiva o si se degrada al añadir más contexto, caso que, como veremos, suele ocurrir con frecuencia.

El funcionamiento general de la estrategia incremental acumulativa se muestra en la Figura 4.3.

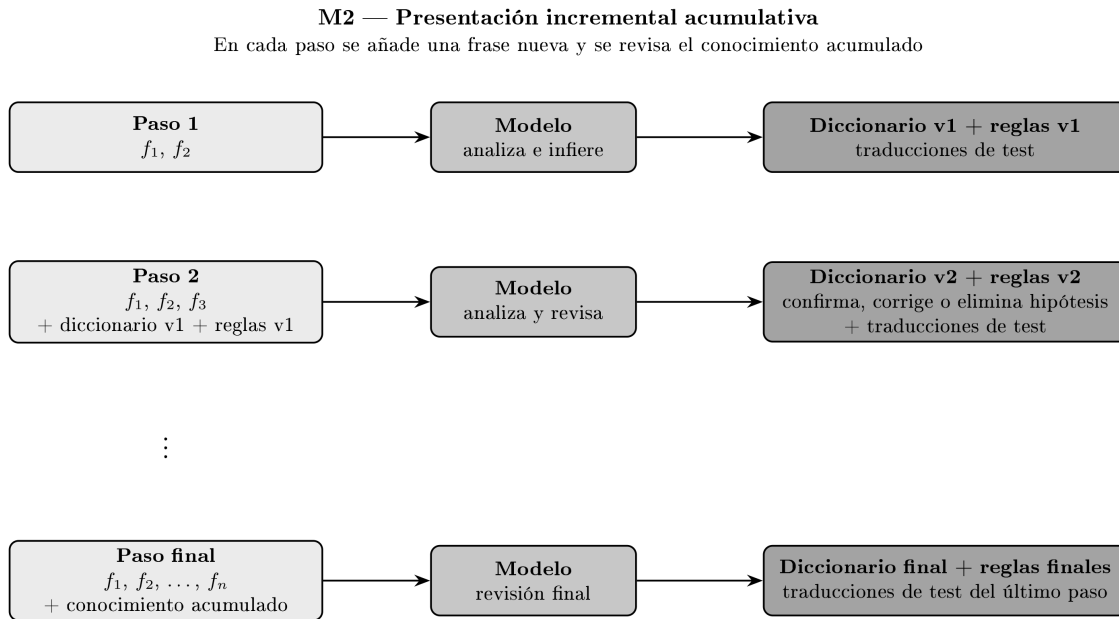


Figura 4.3: Funcionamiento de la estrategia incremental acumulativa M2.

4.3.5. M2.2: Exploración combinatoria por tamaños crecientes

M2.2 se plantea como una extensión de la estrategia incremental. Mientras que M2 utiliza siempre prefijos del conjunto de entrenamiento, M2.2 evaluaría todas las combinaciones posibles de ejemplos para cada tamaño de subconjunto: primero todas las parejas, después todas las ternas, y así sucesivamente hasta llegar al conjunto completo. El objetivo de esta variante sería estudiar si el rendimiento depende del orden y de la selección concreta de los ejemplos. En M2, una hipótesis inicial puede quedar condicionada por las primeras frases del puzle. Con M2.2 sería posible comprobar si otros subconjuntos iniciales conducen a reglas más estables o a mejores traducciones finales.

Sin embargo, como ya se ha indicado en las limitaciones del trabajo, el coste computacional de esta variante crece rápidamente con el número de ejemplos de entrenamiento. Por este motivo, en este trabajo se deja planteada como ampliación metodológica, pero su evaluación completa para todos los puzles seleccionados queda fuera del alcance principal de la entrega. La Figura 4.4 muestra de forma esquemática cómo crecería el número de combinaciones en un caso reducido con cuatro frases de entrenamiento.

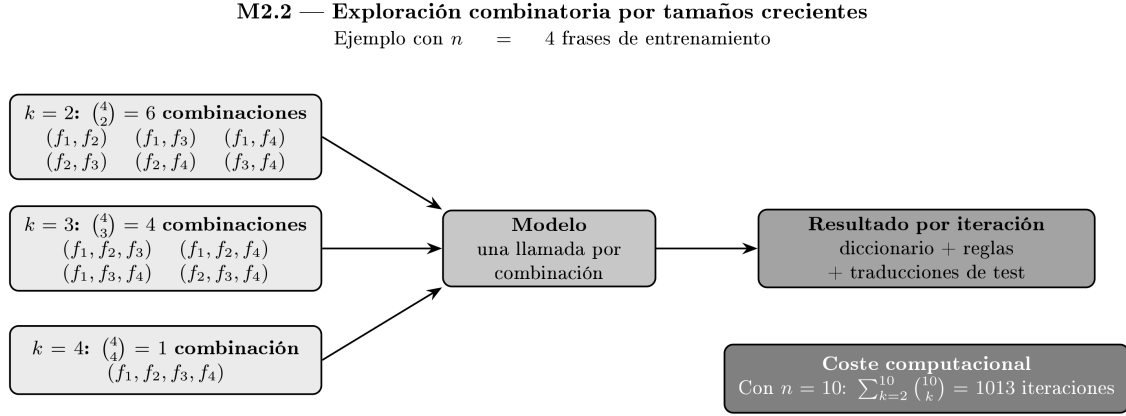


Figura 4.4: Extensión planificada de M2 mediante la exploración exhaustiva de combinaciones de ejemplos.

4.3.6. M3: Step-by-step lingüístico

M3 está inspirada en la estrategia de descomposición lingüística propuesta por Zhu y cols. (2025). En lugar de introducir los ejemplos mediante prefijos crecientes, como en M2, esta metodología organiza el proceso de resolución en cuatro etapas asociadas a niveles de análisis lingüístico: léxico, fonología, morfosintaxis y sintaxis.

La división utilizada por Zhu y cols. (2025) es la siguiente:

1. **Léxico** (35 % de los ejemplos): se presentan ejemplos orientados a identificar vocabulario básico y patrones generales de orden de palabras.
2. **Fonología** (15 % de los ejemplos): se introducen ejemplos que podrían ayudar a detectar variaciones fonológicas, como alternancias vocálicas, cambios consonánticos o formas relacionadas de un mismo morfema.
3. **Morfosintaxis** (30 % de los ejemplos): se añaden ejemplos dirigidos a identificar marcas de persona, número, tiempo, aspecto, caso, concordancia u otros rasgos gramaticales.
4. **Sintaxis** (20 % de los ejemplos): se incorporan los ejemplos restantes para integrar las reglas anteriores y resolver estructuras más complejas.

En cada etapa, el modelo recibe los ejemplos correspondientes junto con el diccionario y las reglas acumuladas en las etapas anteriores. A partir de esa información debe revisar sus hipótesis y traducir las frases de test usando todo el conocimiento disponible hasta ese momento.

Es importante señalar que esta partición es heurística. Los puzles de LINGUINI no incluyen una anotación explícita que indique qué ejemplos corresponden a léxico, fonología, morfosintaxis o sintaxis. Por ello, la asignación se realiza según la posición de los ejemplos en el conjunto de entrenamiento y no garantiza que cada bloque contenga exactamente el fenómeno lingüístico indicado. M3 debe interpretarse, por tanto, como una aproximación guiada

lingüísticamente, no como una descomposición perfecta de cada puzle. El funcionamiento general de esta estrategia se resume en la Figura 4.5.

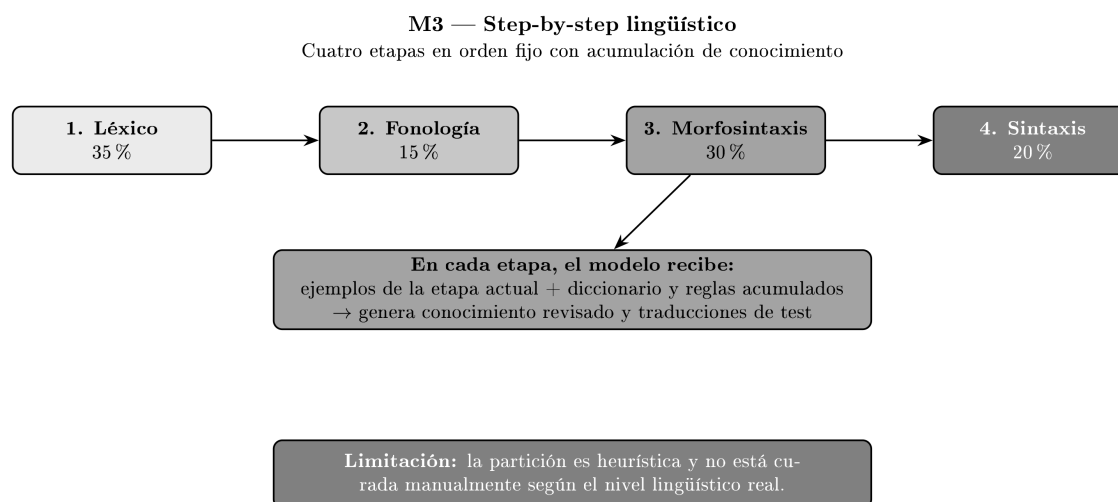


Figura 4.5: Funcionamiento de la estrategia paso a paso lingüística M3.

4.3.7. M4: Inspiración humana

M4 introduce una guía basada en una resolución humana. La idea es proporcionar al modelo el razonamiento explícito de una persona resolviendo un puzle diferente, para que pueda imitar el proceso de análisis sin copiar el contenido lingüístico. Esta estrategia se relaciona con los *think-aloud protocols*, una técnica de análisis cognitivo en la que una persona verbaliza sus pensamientos mientras resuelve una tarea (Ericsson y Simon, 1993). El ejemplo utilizado se construyó a partir de una resolución propia y detallada del puzle de Hankun (véase la Sección 4.6). En esa resolución se muestran los pasos seguidos para identificar elementos invariantes, aislar posibles morfemas, formular hipótesis provisionales y corregirlas a medida que aparecen nuevos datos.

El modelo recibe este razonamiento como referencia de proceso, no como fuente de reglas aplicables al nuevo puzle. Por ello, el prompt indica explícitamente que no debe transferir vocabulario ni reglas del ejemplo humano a la lengua que debe resolver. El objetivo es que reproduzca una forma de analizar: comparar ejemplos, detectar regularidades y revisar hipótesis.

Cabe mencionar que para evitar una fuga directa de información, M4 no se ejecuta sobre ningún puzle de Hankun. De esta manera, el puzle usado como ejemplo humano no coincide con ninguno de los casos evaluados bajo esta estrategia. El funcionamiento general de M4 se resume en la Figura 4.6.

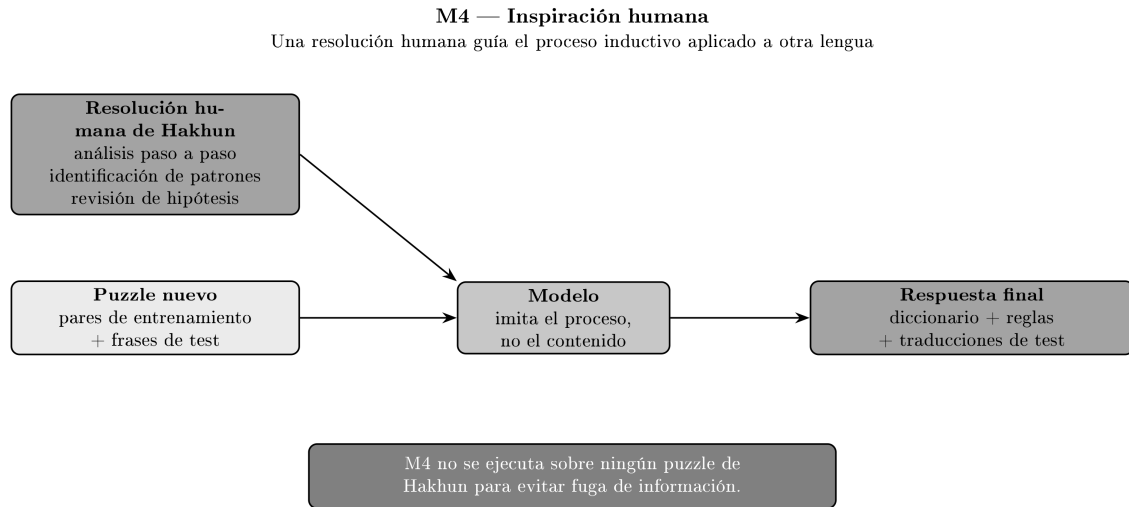


Figura 4.6: Funcionamiento de la estrategia de inspiración humana M4.

4.3.8. M5: Autocorrección iterativa

M5 incorpora una segunda fase de revisión inspirada en el paradigma *Self-Refine* (Madaan y cols., 2023). En el primer turno, el modelo resuelve el puzzle con un prompt equivalente al de M1. Es decir, recibe todos los pares de entrenamiento y genera un diccionario, unas reglas gramaticales y unas traducciones iniciales. En el segundo turno, el modelo recibe su propia respuesta anterior y se le pide que actúe como revisor. La instrucción no se limita a pedir una nueva traducción, sino que obliga al modelo a buscar contradicciones internas entre sus traducciones, su diccionario y las reglas que ha formulado. Para cada posible problema, debe indicar qué regla o entrada de diccionario está siendo violada o no está suficientemente justificada.

La hipótesis detrás de esta estrategia es que algunos errores no se deben a falta de información, sino a incoherencias dentro del propio razonamiento del modelo. Si el modelo ha identificado una regla correctamente pero no la ha aplicado en una traducción concreta, una fase de revisión podría ayudar a corregir el fallo. Al mismo tiempo, esta estrategia permite estudiar una limitación importante: cuando la primera hipótesis es incorrecta, el modelo puede usarla como criterio de revisión y reforzar el error en lugar de corregirlo. El funcionamiento general de M5 se resume en la Figura 4.7.

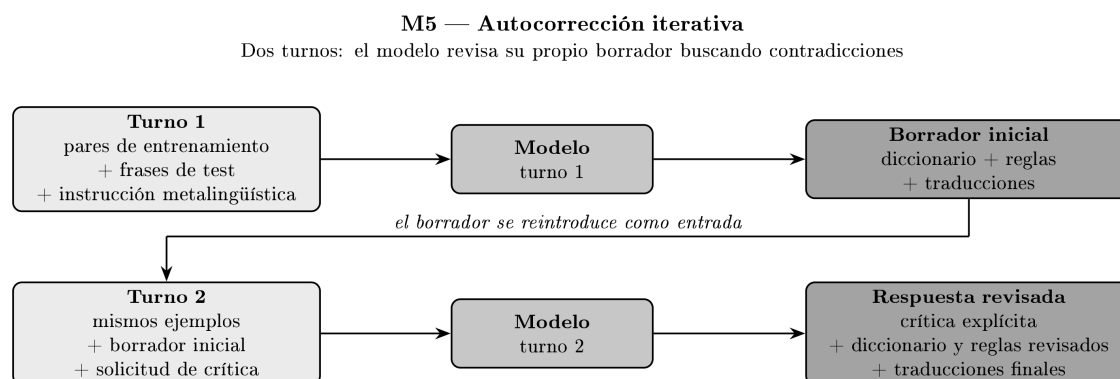


Figura 4.7: Funcionamiento de la estrategia de autocorrección iterativa M5.

4.4. Sistema experimental

Para ejecutar los experimentos se desarrolló un sistema experimental en Python, organizado en torno al script `experiment_runner.py`. Su función principal es automatizar el proceso de evaluación: cargar un puzzle, generar el prompt correspondiente a cada estrategia, enviar la petición al modelo, extraer las traducciones producidas, calcular las métricas de evaluación descritas en la Sección 4.5 y guardar los resultados en un archivo estructurado. El sistema permite ejecutar de forma homogénea el control M0 y las estrategias M1–M5 sobre los distintos puzzles seleccionados. En las estrategias iterativas, como M2 y M3, también registra los pasos intermedios, incluyendo el diccionario y las reglas que el modelo va formulando. Esto permite analizar no solo el resultado final, sino también la evolución del razonamiento durante la resolución. Cada experimento genera un log en formato JSON con la información necesaria para reproducir y revisar los resultados: modelo utilizado, puzzle evaluado, estrategia aplicada, traducciones extraídas, métricas obtenidas y, cuando corresponde, hipótesis acumuladas en cada paso. Esta estructura facilita la comparación entre metodologías y permite auditar manualmente casos concretos sin repetir las llamadas a las APIs.

Los detalles de implementación del sistema, incluyendo el formato de entrada, el diseño de los prompts, el extractor de traducciones y los problemas técnicos encontrados durante el desarrollo, se describen de forma más detallada en el Capítulo 5.

4.5. Métricas de evaluación

La evaluación automática se realiza mediante cuatro métricas: BLEU, chrF++, TER y Exact Match. Todas ellas se calculan comparando las traducciones generadas por el modelo con las referencias correctas del benchmark. El uso conjunto de varias métricas permite observar distintos tipos de error, ya que una única medida puede ser insuficiente en frases cortas o en lenguas con morfología compleja.

BLEU (Papineni y cols., 2002) mide la coincidencia de n-gramas entre la traducción generada y la referencia, incorporando una penalización por brevedad. De forma simplificada, BLEU puede expresarse como:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

donde p_n representa la precisión de los n -gramas de tamaño n , w_n es el peso asignado a cada tamaño de n -grama y BP es la penalización por brevedad. Esta penalización evita que una traducción demasiado corta obtenga una puntuación alta solo por contener algunas palabras correctas. Por ejemplo, si la referencia es *the boy eats rice* y el modelo responde únicamente *the boy*, hay coincidencias parciales, pero la respuesta es incompleta y BLEU debe penalizarla. Es una métrica muy utilizada en traducción automática, pero en este trabajo tiene una limitación clara: las frases de test suelen ser cortas y pequeñas diferencias morfológicas o léxicas pueden producir puntuaciones muy bajas. Se mantiene principalmente por compatibilidad con la literatura previa ($\uparrow$ mejor, rango 0–100).

Como aclaración, la flecha hacia arriba en las métricas ($\uparrow$) significa que cuanto más alto es el valor, más acertado es el resultado. En caso contrario, es decir, una flecha hacia abajo ($\downarrow$) significa que cuanto más bajo, mejor.

chrF++ (Popović, 2017) es la métrica principal del trabajo. A diferencia de BLEU, opera principalmente a nivel de n -gramas de caracteres e incorpora también información de palabras mediante `word_order=2`. Este parámetro indica que, además de comparar fragmentos de caracteres, la métrica también tiene en cuenta n -gramas de palabras hasta orden 2, es decir, palabras individuales y pares de palabras consecutivas. Por ejemplo, entre *the big dog* y *the dog big* habría muchas coincidencias de caracteres y de palabras individuales, pero no coincidirían igual los pares de palabras consecutivas. De esta forma, `word_order=2` permite reflejar parcialmente diferencias en el orden de las palabras. La métrica se basa en una media F , que combina precisión y cobertura:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$$

En este contexto, la precisión mide qué parte de los fragmentos generados por el modelo aparece también en la referencia, mientras que la cobertura mide qué parte de la referencia ha sido recuperada por la traducción generada. Esto la hace más adecuada para este tipo de tarea, ya que puede reflejar similitudes parciales incluso cuando la traducción no coincide exactamente con la referencia. Por ejemplo, si la referencia contiene *the children* y el modelo genera *the child*, Exact Match marcaría la respuesta como incorrecta, pero chrF++ todavía captaría una similitud alta entre ambas formas. Resulta especialmente útil en lenguas con morfología rica o cuando los errores afectan a partes pequeñas de una palabra ($\uparrow$ mejor, rango 0–100).

TER (Snover y cols., 2006) (*Translation Edit Rate*) mide el número de ediciones necesarias para transformar la hipótesis del modelo en la referencia. De forma general, se calcula como:

$$TER = \frac{\text{número de ediciones}}{\text{número de palabras de la referencia}}$$

Las ediciones pueden incluir inserciones, eliminaciones, sustituciones o desplazamientos de palabras. A diferencia de BLEU y chrF++, es una métrica de error: cuanto menor es el valor, mejor es la traducción. Un valor de 0 indica coincidencia perfecta con la referencia, mientras

que valores altos indican que la respuesta requiere muchas modificaciones. Por ejemplo, si la referencia es *the woman sees the dog* y el modelo responde *the man sees the dog*, bastaría una sustitución para acercar la hipótesis a la referencia, por lo que el error sería relativamente bajo. En cambio, una traducción completamente distinta requeriría muchas más ediciones ($\downarrow$ mejor).

Exact Match (EM) mide el porcentaje de traducciones exactamente correctas después de una normalización básica, como pasar a minúsculas y eliminar puntuación final. En este trabajo se calcula como:

$$EM = \frac{\text{número de respuestas exactamente correctas}}{\text{número total de respuestas}} \cdot 100$$

Es una métrica estricta, pero fácil de interpretar: una respuesta cuenta como correcta solo si coincide con la referencia. Por ejemplo, si la referencia es *the dog sleeps* y el modelo responde exactamente *the dog sleeps*, la respuesta cuenta como correcta. En cambio, si responde *a dog sleeps*, se considera incorrecta aunque el significado sea muy parecido. En este trabajo debe interpretarse con cautela, porque cada puzzle contiene pocas frases de test; por tanto, una sola respuesta correcta o incorrecta puede cambiar bastante el porcentaje final ($\uparrow$ mejor, rango 0–100 %).

En el análisis de resultados se toma chrF++ como métrica principal, ya que ofrece un equilibrio razonable entre coincidencia exacta y similitud parcial. BLEU, TER y Exact Match se utilizan como métricas complementarias para interpretar mejor los resultados y detectar casos en los que una traducción es parcialmente correcta, pero no coincide completamente con la referencia.

4.6. Análisis cualitativo

Además de la evaluación automática, se realiza un análisis cualitativo sobre el puzzle de Hankun. El objetivo es comparar el proceso de resolución humana con el razonamiento generado por el modelo, prestando atención a aspectos que las métricas automáticas no capturan bien, como la formulación de hipótesis, la identificación de morfemas o la aplicación coherente de reglas gramaticales.

El puzzle fue resuelto manualmente por mí, como autora de este trabajo, anotando de forma explícita el razonamiento seguido durante la deducción. Esta resolución propia se utiliza como referencia cualitativa para observar en qué puntos el comportamiento del modelo se aproxima al razonamiento humano y en cuáles aparecen divergencias.

4.6.1. Proceso de resolución humana

El protocolo consistió en resolver el personalmente puzzle, anotando explícitamente cada paso del razonamiento: qué patrones se identificaron primero, qué hipótesis se formularon, cuándo y por qué se corrigieron, y cómo se aplicaron a las frases de test.

El proceso siguió el siguiente orden: primero se identifican los elementos invariantes (**ne** como marcador de pregunta, presente en todas las frases), luego los pronombres de sujeto (**ŋa**=I, **nɤ**=you(sg), **nirum**=we, **tarum**=they), después los verbos (**lapk^{hi}**=see, **c^ham**=know, **lan**=beat, **ka**=go, **ʒip**=sleep), y finalmente el sistema aspectual: las terminaciones con ? o

t^h- marcan pasado, mientras que las demás marcan presente. El marcador *kəmə* se identificó como marcador de transitividad sin traducción directa al inglés. Aquí muestro mi proceso de resolución, con mi propio razonamiento y mis palabras:

Tabla 4.3: Pares de entrenamiento del puzle de Hankun.

N.º	Hankun	English
1	ŋa ka kɤ ne	Do I go?
2	nɤ ʒip tuʔ ne	Did you sleep?
3	ŋabə ati lapk ^{hi} tɤʔ ne	Did I see him?
4	nirum kəmə nuʔrum c ^h am ki ne	Do we know you?
5	nɤbə ŋa lapk ^{hi} rɤ ne	Do you see me?
6	tarum kəmə nɤ lan t ^h u ne	Did they beat you?
7	nuʔrum kəmə ati lapk ^{hi} kan ne	Do you see him?
8	nɤbə ati c ^h am tuʔ ne	Did you know him?
9	tarum kəmə nirum lapk ^{hi} ri ne	Do they see us?
10	ati kəmə ŋa lapk ^{hi} t ^h ɤ ne	Did he see me?

Frase 1: nɤ ʒip ku ne

- Puedo observar que *ne* se utiliza en todas las frases al final, lo que me hace pensar que puede ser la propia pregunta *Do* o *Did*, o una forma de interpretar el signo de interrogación.
- También hay un patrón con *ŋa*, ya que siempre lo usa en referencia a la primera persona (yo, a mí...), igual que *nɤ* en segunda persona singular.
- Por ahora, la estructura sería algo similar a: “Did you ...?”.
- *ʒip* solo aparece una vez en las traducciones, por lo que seguramente sea dormir (*sleep*).
- Falta interpretar *ku*, que por la similitud que observo con la segunda frase, deduzco que este símbolo (?) hace referencia al pasado (*did*), mientras que el resto están en presente.
- **Mi conclusión:** Do you(sg) sleep?

Frase 2: ati kəmə nirum lapk^{hi} t^hi ne

- Observando frases con *ati*, probablemente se traduzca como *he/him*, dependiendo del contexto.
- *kəmə* aparece en varias frases, pero no tiene un patrón claro; parece más un marcador de objeto, por ahora lo obviaremos.
- *nirum* es la primera persona en plural.
- *lapk^{hi}* es claramente el verbo ver (*see*).

- t^{hi} seguramente apunte al pasado, ya que en todas las frases referentes a acciones pasadas aparece el símbolo.
- **Mi conclusión:** Did he see us?

Frase 3: tarum kəmə nuʔrum c^ham ran ne

- tarum hace referencia clara a ellos (*they*); kəmə lo vamos a dejar como marcador sin traducción directa.
- He visto que la primera persona en plural es nirum; nuʔrum parece ser la segunda persona en plural, aunque este símbolo me hace dudar de la estructura: ʔ.
- c^ham es claramente el verbo saber (*know*).
- ran no está en ninguna parte, pero sí kan; por lo que seguramente deba apuntar al presente. No es pasado.
- **Mi conclusión:** Do they know you(pl)?

Frase 4: nirum kəmə tarum lan ki ne

- nirum es primera persona en plural claramente.
- tarum es ellos.
- kəmə es un marcador.
- lan seguramente sea el verbo; aparece una sola vez para traducir *beat*.
- Con lo que he visto, ki debe ser el tiempo presente.
- **Mi conclusión:** Do we beat them?

Frase 5: nirum kəmə nɣ c^ham tiʔ ne

- nirum = nosotros (*we*).
 - nɣ = tú (*you sg*).
 - c^ham = saber (*know*).
 - Aquí aparece tiʔ, por lo que es pasado seguro.
 - **Mi conclusión:** Did we know you(sg)?
-

Frase 6: nirum ka ti? ne

- Aquí tenemos dos cosas claras:
- nirum = we.
- ti? = tiempo pasado.
- No tengo marcador de objeto **kəmə**, supongo que nadie realiza la acción hacia nadie.
- ka aparece una vez para referirse al verbo ir (*go*).
- **Mi conclusión:** Did we go?

4.6.2. Comparación con el modelo

Como se ha comentado, además de la evaluación automática, se plantea una comparación cualitativa entre la resolución humana utilizada como guía en M4 y las respuestas generadas por los modelos. El puzle de Hankun fue resuelto manualmente por mí, como autora de este trabajo, anotando de forma explícita los pasos seguidos durante la deducción. Esta resolución propia se utiliza como referencia cualitativa para analizar en qué puntos el razonamiento del modelo se aproxima al proceso humano y en cuáles aparecen divergencias. El análisis detallado se presenta junto con los resultados de M4, en la sección 6.5.1.

5. Desarrollo del sistema

Este capítulo describe la implementación del sistema experimental utilizado para ejecutar y evaluar las distintas estrategias de prompting. En concreto, se explica la arquitectura en dos fases, el formato de entrada y salida, el diseño de los prompts, el extractor de traducciones, el proceso de construcción del conjunto de datos y los principales problemas técnicos encontrados durante el desarrollo.

5.1. Arquitectura general

El sistema se implementó en Python e inicialmente se organizó con un script que directamente extrae el texto con las hipótesis de traducción que devuelve el modelo. Pero debido a que el modelo no siempre responde como le pedimos (en este caso, con una estructura JSON que divida las frases traducidas en bloques para que sean más sencillas de rescatar), algunas extracciones podían no ser del todo fiables. Finalmente, para poder verificar la respuesta completa del modelo, se terminaron implementando dos scripts independientes. Esta separación fue una de las decisiones más importantes del desarrollo, ya que permite distinguir claramente entre la captura de respuestas del modelo y el cálculo posterior de métricas.

experiment_runner.py Gestiona la comunicación con las APIs, construye los prompts de cada estrategia y guarda en disco la respuesta completa del modelo antes de aplicar ningún procesamiento adicional. El resultado de esta fase es el log crudo del experimento.

rescore_from_raw.py Lee los logs crudos, aplica el extractor de traducciones y calcula las métricas. Como no realiza llamadas a la API, puede ejecutarse tantas veces como sea necesario, y este punto es importante ya que de esta manera se podría corregir un error del extractor mientras se verificaba que los resultados son fiables, sin consumir cuota adicional.

En las primeras pruebas, algunos errores no procedían de la respuesta del modelo, sino de la forma en que el sistema extraía las traducciones finales. Si la extracción se hubiese realizado directamente durante la llamada a la API, habría sido necesario repetir muchas ejecuciones. Al conservar siempre la respuesta original, fue posible corregir el extractor y recalcular las métricas a partir de los mismos datos.

5.1.1. Formato de entrada: el puzzle

Cada puzzle se almacena en un archivo JSON independiente. El archivo contiene dos listas diferenciadas: **train_pairs**, que se incorporan al prompt como ejemplos de entrenamiento, y **test_pairs**, que se reservan para la evaluación. Las traducciones correctas de los pares de test no se muestran nunca al modelo durante la inferencia.

Código 5.1: Estructura de un puzzle (ejemplo: Hankun lang2en)

```

1 {
2   "id": "linguini_012018020100",
3   "language": "Hankun",
4   "language_family": "Sino-Tibetan",
5   "translation_direction": "lang2en",
6   "train_pairs": [
7     { "source": "n ip tu ne", "target": "Did you sleep?" },
8     { "source": "ati ækm nirum lapki ti ne",
9       "target": "Did he see us?" }
10  ],
11  "test_pairs": [
12    { "source": "n ip ku ne", "target": "Do you sleep?" },
13    { "source": "ati ækm nirum lapki ran ne",
14      "target": "Did he see us?" }
15  ]
16 }

```

5.1.2. Formato del log crudo

Cada llamada al modelo queda registrada en un archivo `.jsonl`, con una línea JSON por llamada. El registro se realiza justo después de recibir la respuesta, antes de que actúe el extractor. De esta forma, cualquier fallo posterior en el procesamiento no provoca la pérdida del texto generado por el modelo y además, en caso de problema, permite ver el log crudo para extraer incluso las traducciones manualmente.

Código 5.2: Registro de una llamada al modelo en `raw_logs/` (respuesta abreviada)

```

1 {
2   "timestamp": "2026-06-22T10:15:32",
3   "strategy": "baseline",
4   "step": 1,
5   "prompt": "You are an expert linguistic analyst...\n
6   --- TRAINING EXAMPLES ---\n
7   n ip tu ne → Did you sleep?\n
8   ...\n
9   --- YOUR TASK ---\n
10  Translate into English:\n
11  1. n ip ku ne\n...",
12  "response": "DICTIONARY:\nɲna = I \ nn = you \n
13  ati = he/him\n...\n
14  GRAMMAR RULES:\n
15  1. Sentences end with 'ne' (question marker).\n
16  2. Past tense: suffix tu/ ti; present: ku/ran.\n...\n
17  TRANSLATIONS:\n
18  1. Do you sleep?\n
19  2. Did he see us?\n
20  3. Do they know you?\n
21  4. Do we beat them?\n
22  5. Did we know you?\n

```

```

23 6. Did we go?"
24 }

```

5.1.3. Formato del JSON de métricas

La salida de `rescore_from_raw.py` contiene, para cada estrategia, los campos `primary_metrics`, `best_metrics` y la lista de pasos con sus métricas individuales. El campo `primary_metrics` corresponde al resultado que se reporta en las tablas principales. En cambio, `best_metrics` se conserva únicamente como información diagnóstica, especialmente en estrategias iterativas donde puede interesar saber si algún paso intermedio obtuvo mejor resultado que el paso final. Como veremos, a veces estos dos campos no coinciden debido a que el modelo pueda devaluar la respuesta en los últimos pasos.

El Código 5.3 muestra un extracto real del puzzle de Hakhun con la estrategia M1, que además, destaca por haber sido la única salida perfecta de todo el experimento.

Código 5.3: Extracto del JSON de métricas para Hakhun M1 (GPT-OSS-120B). Los valores son los obtenidos en el experimento con normalización aplicada

```

1 {
2   "metadata": {
3     "puzzle_id": "linguini_012018020100",
4     "language": "Hakhun",
5     "model_key": "gpt-oss-120b-openrouter",
6     "num_test_pairs": 6
7   },
8   "results": {
9     "baseline": {
10      "primary_metrics": {
11        "corpus_chrfpp": 100.0,
12        "corpus_bleu": 100.0,
13        "corpus_ter": 0.0,
14        "exact_match_corpus": 100.0,
15        "complete": true,
16        "missing_indices": [],
17        "num_evaluated": 6,
18        "num_expected": 6,
19        "hypotheses": [
20          "Do you sleep?",
21          "Did he see us?",
22          "Do they know you?",
23          "Do we beat them?",
24          "Did we know you?",
25          "Did we go?"
26        ],
27        "references": [
28          "Do you sleep?",
29          "Did he see us?",
30          "Do they know you?",
31          "Do we beat them?",
32          "Did we know you?",
33          "Did we go?"

```

```

34],
35"sentence_chrfpp": [100.0, 100.0, 100.0, 100.0, 100.0, 100.0]
36}
37}
38}
39}

```

Este extracto permite ver cómo se compara cada hipótesis generada por el modelo con su referencia. Las seis frases obtienen chrF++=100 porque coinciden exactamente con la referencia tras la normalización. Cabe notar que las referencias originales del dataset Linguini incluyen marcadores de número gramatical (sg) y (p1) —por ejemplo, you(sg) o you(p1)— que son anotaciones propias del corpus y no forman parte del inglés natural. Dado que los modelos producen inglés sin estos marcadores (lo cual es correcto), incluirlos en la evaluación penalizaría artificialmente las traducciones correctas. Por decisión final, se implementó una función de normalización (`normalize_reference`) que elimina estos marcadores de **ambos lados** —hipótesis y referencia— antes de calcular cualquier métrica, garantizando que la evaluación mida únicamente la calidad lingüística de la traducción.

Para estrategias iterativas como M2, el JSON incluye además la evolución paso a paso. El siguiente extracto muestra cómo crece la calidad de Hakhun a lo largo de los 9 pasos de la estrategia incremental:

Código 5.4: Evolución del chrF++ por paso en M2 para Hakhun (GPT-OSS-120B).

```

1 "incremental_combinations": {
2   "primary_step": 9,
3   "primary_rule": "last",
4   "primary_metrics": { "corpus_chrfpp": 91.7 },
5   "best_metrics": { "corpus_chrfpp": 91.7 },
6   "steps": [
7     { "step": 1, "metrics": { "corpus_chrfpp": 14.3 } },
8     { "step": 2, "metrics": { "corpus_chrfpp": 20.0 } },
9     { ... },
10    { "step": 9, "metrics": { "corpus_chrfpp": 91.7 } }
11  ]
12 }

```

El campo `primary_rule`: "last" indica que se reporta el último paso, es decir, el momento en que el modelo ya ha recibido todos los ejemplos de entrenamiento. No se reporta automáticamente el paso con mayor puntuación, ya que eso introduciría un sesgo favorable al utilizar las respuestas de test para escoger el mejor resultado, además de no ser un resultado real, ya que éste se debe extraer siempre de la conclusión final. Esta decisión se resume junto con el resto de decisiones técnicas en la Tabla 5.2, situada en el final de este capítulo. El paso máximo se conserva en `best_metrics` únicamente como diagnóstico de estabilidad.

5.2. Diseño de los prompts

Una vez vistas, en el apartado de metodología todas las estrategias de prompting de forma teórica, procedemos a observar las plantillas de prompt utilizadas por el sistema experimental. En lugar de mostrar ejemplos simplificados, se incluyen las estructuras completas empleadas

en el script de lanzamiento. Algunas partes se generan dinámicamente en función del puzzle, como los pares de entrenamiento, las frases de test, la dirección de traducción o el número exacto de respuestas esperadas.

5.2.1. Instrucción metalingüística base

Todas las estrategias principales, de M1 a M5, comienzan con una misma instrucción metalingüística. Su objetivo es fijar el papel del modelo como analista lingüístico y recordarle que debe razonar únicamente a partir de los ejemplos proporcionados.

Código 5.5: Instrucción metalingüística base utilizada en M1–M5.

```

1 You are an expert linguistic analyst performing inductive reasoning on an unknown language.
2 You have never seen this language before. You must reason exclusively from the provided
   ↪ examples using your meta-linguistic knowledge of how human languages work in ↪
   ↪ general (word order typology, morphological patterns, agreement systems, ↪
   ↪ affixation, etc.).
3
4 Core principles:
5 – Do NOT use any prior knowledge of this specific language or its family
6 – Treat every hypothesis as provisional and subject to revision
7 – Always show your reasoning explicitly before reaching any conclusion
8 – A wrong rule confidently stated is worse than uncertainty acknowledged

```

Los prompts se redactan en inglés porque es la lengua en la que los modelos suelen mostrar mejor rendimiento en tareas de razonamiento, y porque las traducciones de los puzzles tienen siempre el inglés como lengua origen o destino. Además, la instrucción de no utilizar conocimiento previo es importante porque algunos modelos pueden haber visto información sobre determinadas lenguas durante su preentrenamiento.

5.2.2. Bloque dinámico de traducciones

Todas las estrategias que producen traducciones utilizan un bloque final **TRANSLATIONS**. Este bloque se genera dinámicamente con el número real de frases de test del puzzle. Así se evita mostrar al modelo un ejemplo genérico con solo dos líneas cuando, en realidad, debe responder tres, cuatro o seis frases.

Código 5.6: Bloque dinámico de salida para n frases de test.

```

1 TRANSLATIONS:
2 1. [translation only]
3 2. [translation only]
4 ...
5 n. [translation only]
6 === END ===
7
8 You MUST provide all n numbered lines above, from 1 to n, in that exact
9 order. Never skip a number: if you are unsure, give your best guess anyway.
10 Write nothing after === END ===.

```

El marcador `=== END ===` se añadió posteriormente para facilitar el bucle en el código de la

extracción automática (como una marca de dónde debe terminar la sección de traducciones), ya que algunos modelos añadían explicaciones después de las traducciones, y esas notas también se interpretaban como respuestas finales.

5.2.3. Prompt de M0: verificación previa

M0 no utiliza la instrucción metalingüística base. Su objetivo es comprobar si el modelo puede traducir una frase de test sin haber recibido ningún ejemplo de entrenamiento. A esto le llamamos la verificación de data contamination.

Para puzzles en dirección lengua desconocida → inglés se utiliza:

Código 5.7: Prompt de M0 para dirección lengua desconocida → inglés.

```
1 Translate the following sentence into English. Give your best attempt even if you are not sure.
2
3 Sentence: {sentence}
4
5 Translation:
```

Para puzzles en dirección inglés → lengua desconocida, la plantilla cambia la lengua de destino:

Código 5.8: Prompt de M0 para dirección inglés → lengua desconocida.

```
1 Translate the following sentence into {language}. Give your best attempt even if you are not ↔
   ↔ sure.
2
3 Sentence: {sentence}
4
5 Translation:
```

En ambos casos, {sentence} se sustituye por la primera frase del conjunto de test para cada problema a analizar. La traducción generada se compara con la referencia mediante chrF++. Si el valor supera el umbral definido (en este caso es 20), el puzzle se marca para revisión manual. En el caso de las pruebas realizadas para este proyecto, todos los puzzles devolvieron un valor inferior al límite.

5.2.4. Prompt de M1: baseline

M1 presenta todos los pares de entrenamiento en un único prompt. Es la estrategia más directa y sirve como punto de comparación para las demás metodologías.

Código 5.9: Plantilla completa del prompt de M1.

```
1 {METALINGUISTIC_INSTRUCTION}
2
3 ---- TRAINING EXAMPLES ----
4 {examples}
5
6 ---- YOUR TASK ----
7 Based on ALL examples above, write your inferred DICTIONARY and GRAMMAR RULES,
8 then {translation_instruction}:
```

```

9 {tests}
10
11 Format your response EXACTLY as:
12 DICTIONARY:
13 [word mappings, one per line]
14
15 GRAMMAR RULES:
16 [rules, one per line]
17
18 {TRANSLATIONS_BLOCK}

```

En esta plantilla, `{examples}` contiene todos los pares de entrenamiento del puzzle en formato **fuente** -> **traducción**. El campo `{tests}` contiene las frases que el modelo debe traducir, numeradas en el mismo orden en que serán evaluadas. Por último, `{TRANSLATIONS_BLOCK}` se sustituye por el bloque dinámico descrito en la Sección 5.2.2.

5.2.5. Prompt de M2: presentación incremental acumulativa

M2 utiliza un prompt distinto en cada paso. En lugar de mostrar todos los ejemplos desde el principio, introduce subconjuntos crecientes y proporciona al modelo la hipótesis acumulada del paso anterior. El objetivo es que el modelo revise su diccionario y sus reglas a medida que recibe nueva evidencia.

Código 5.10: Plantilla completa del prompt de M2.

```

1 {METALINGUISTIC_INSTRUCTION}
2
3 STEP {step}/{total} — New subset of examples:
4
5 --- YOUR CURRENT HYPOTHESIS (provisional — review critically) ---
6 DICTIONARY:
7 {current_dictionary}
8
9 GRAMMAR RULES:
10 {current_rules}
11
12 IMPORTANT: New examples may CONFIRM, REFINE or CONTRADICT entries above.
13 Delete proven—wrong entries; correct imprecise ones.
14
15 --- NEW EXAMPLES ---
16 {new_examples}
17
18 --- YOUR TASK (in this order) ---
19
20 STEP A · ANALYSIS
21 Identify morphemes in each new example. Check for contradictions with your
22 current hypothesis. Think aloud. Do NOT translate the test sentences yet.
23
24 STEP B · REVISED DICTIONARY
25 One entry per line: morpheme = meaning (notes)
26 Mark deleted entries: [DELETED: entry — reason]

```

```

27
28STEP C · REVISED GRAMMAR RULES
29Numbered list. Mark deleted rules: [DELETED: rule — reason]
30
31STEP D · TRANSLATIONS
32{translation_instruction} — write ONLY the translation on each line, nothing else:
33{tests}
34
35Format your response EXACTLY as:
36ANALYSIS:
37[reasoning]
38
39DICTIONARY:
40[entries]
41
42GRAMMAR RULES:
43[rules]
44
45{TRANSLATIONS_BLOCK}

```

En el primer paso, cuando todavía no existe conocimiento acumulado, el bloque de hipótesis previa se sustituye por una indicación de que no hay conocimiento anterior. En los pasos siguientes, {current_dictionary} y {current_rules} contienen el diccionario y las reglas extraídas del paso previo. Esta estructura obliga al modelo a tratar sus hipótesis como revisables, no como conclusiones definitivas.

5.2.6. Prompt de M3: step-by-step lingüístico

M3 divide los ejemplos en cuatro etapas: léxico, fonología, morfosintaxis y sintaxis. Cada etapa utiliza el mismo formato general, pero cambia el foco lingüístico que se le indica al modelo.

Código 5.11: Plantilla completa del prompt de M3.

```

1 {METALINGUISTIC_INSTRUCTION}
2
3 STEP {step}/4 — Stage: {stage}
4 {stage_focus}
5
6 --- ACCUMULATED KNOWLEDGE ---
7 DICTIONARY:
8 {current_dictionary}
9
10 GRAMMAR RULES:
11 {current_rules}
12
13 --- EXAMPLES FOR THIS STAGE ---
14 {stage_examples}
15
16 --- YOUR TASK ---
17 1. Refine DICTIONARY with new vocabulary

```

```

18 2. Add/update GRAMMAR RULES for this stage
19 3. {translation_instruction} using ALL accumulated knowledge
20
21 Format your response EXACTLY as:
22 DICTIONARY:
23 [word mappings]
24
25 GRAMMAR RULES:
26 [rules]
27
28 {TRANSLATIONS_BLOCK}

```

Los focos lingüísticos utilizados en cada etapa son los siguientes:

Código 5.12: Focos lingüísticos utilizados en M3.

```

1 LEXICAL:
2 Focus on vocabulary and basic word order.
3
4 PHONOLOGY:
5 Focus on phonological patterns: vowel changes, consonant alternations, affixes, allomorphs.
6
7 MORPHOSYNTAX:
8 Focus on agreement, tense, number, person, gender markings.
9
10 SYNTAX:
11 Combine all rules: negation, questions, complex clauses.

```

Aunque el formato está inspirado en una progresión lingüística, la partición de ejemplos es heurística. Los puzzles de LINGUINI no indican qué frase ilustra cada fenómeno lingüístico ni separan explícitamente los ejemplos por vocabulario, morfología, sintaxis o traducción completa. Por este motivo, el sistema reparte los pares de entrenamiento según porcentajes aproximados: una primera parte se utiliza para identificar vocabulario básico, una segunda parte para observar posibles regularidades morfológicas, una tercera para formular reglas sintácticas o gramaticales más generales y, finalmente, el conjunto completo se usa para producir las traducciones de test.

Esta división no pretende afirmar que cada bloque contenga únicamente ese tipo de información, ya que en un puzzle real una misma frase puede aportar simultáneamente vocabulario, marcas morfológicas y orden sintáctico. Se trata más bien de una forma de guiar progresivamente al modelo, obligándolo a construir hipótesis parciales antes de llegar a la traducción final. Por tanto, M3 debe entenderse como una estrategia de estructuración del razonamiento, no como una segmentación lingüística exacta del dataset.

5.2.7. Prompt de M4: inspiración humana

M4 añade antes del puzzle nuevo una resolución humana completa del puzzle de Hakhun. Esta resolución no se utiliza como fuente de vocabulario ni de reglas para la nueva lengua, sino como ejemplo del tipo de razonamiento que se quiere inducir.

Código 5.13: Cabecera del razonamiento humano utilizado en M4.

```

1 The following is a real human's reasoning while translating an unknown language puzzle.
2 Use it as inspiration for your OWN reasoning process on a DIFFERENT language below.
3 Do NOT copy vocabulary or rules — they belong to a completely different language.
4
5 ---- HUMAN REASONING EXAMPLE (Hakhun language, Sino-Tibetan) ----

```

Después de esta cabecera, el sistema inserta el razonamiento humano completo frase a frase. El contenido incluye la identificación de **ne** como marcador interrogativo, el análisis de los pronombres, la deducción del sistema aspectual y la identificación de verbos como **lapk i**, **c am**, **lan**, **ka/k** y **ip**. El bloque termina con un resumen de los patrones encontrados.

Código 5.14: Resumen final del razonamiento humano insertado en M4.

```

1 Key patterns identified:
2 - Tense suffixes: tu/ t/ t/ tu/ ti/ ti = past (Did); ku/ki/ r/ri/ran/kan = present (Do)
3 - Pronouns: ŋa=I, n=you(sg), ati=he, nirum=we, nurum=you(pl), tarum=they
4 - 'ækm' = transitive object marker
5 - 'lapki' = see, 'cam' = know, 'lan' = beat, 'ka/ k' = go, 'ip' = sleep
6 - 'ne' = sentence-final question marker
7 ---- END OF HUMAN EXAMPLE ----

```

A continuación, el prompt presenta el nuevo puzzle y recuerda explícitamente que no se debe transferir contenido lingüístico del ejemplo de Hakhun:

Código 5.15: Plantilla completa del prompt de M4 tras insertar el razonamiento humano.

```

1 {METALINGUISTIC_INSTRUCTION}
2
3 {HUMAN_RESOLUTION_HAKHUN}
4
5 Now apply the same style of reasoning to a NEW and COMPLETELY DIFFERENT ↵
   ↵ linguistic puzzle.
6 The language below is unrelated to the example above — do not transfer any vocabulary or ↵
   ↵ rules.
7
8 ---- TRAINING EXAMPLES (new language) ----
9 {examples}
10
11 ---- YOUR TASK ----
12 Following the same step-by-step reasoning shown in the human example,
13 write your DICTIONARY and GRAMMAR RULES, then {translation_instruction}:
14 {tests}
15
16 Format your response EXACTLY as:
17 DICTIONARY:
18 [word mappings, one per line]
19
20 GRAMMAR RULES:
21 [rules, one per line]
22
23 {TRANSLATIONS_BLOCK}

```

Por supuesto, cabe mencionar de nuevo que para evitar la fuga de información, M4 no se

ejecuta sobre el puzzle de Hakhun.

5.2.8. Prompt de M5: segundo revisor

M5 trabaja en dos turnos. El primer turno usa el mismo prompt que M1. El segundo turno devuelve al modelo su propia respuesta inicial y le pide que actúe como revisor.

Código 5.16: Prompt del turno 1 de M5.

```

1 {METALINGUISTIC_INSTRUCTION}
2
3 ---- TRAINING EXAMPLES ----
4 {examples}
5
6 ---- YOUR TASK ----
7 Based on ALL examples above, write your inferred DICTIONARY and GRAMMAR RULES,
8 then {translation_instruction}:
9 {tests}
10
11 Format your response EXACTLY as:
12 DICTIONARY:
13 [word mappings, one per line]
14
15 GRAMMAR RULES:
16 [rules, one per line]
17
18 {TRANSLATIONS_BLOCK}

```

El segundo turno incluye el diccionario, las reglas y las traducciones generadas en el primer turno. La instrucción de crítica obliga al modelo a comprobar sus propias respuestas frente a los ejemplos de entrenamiento.

Código 5.17: Prompt completo del turno 2 de M5.

```

1 {METALINGUISTIC_INSTRUCTION}
2
3 You have already analyzed the following language and produced a first draft.
4 Now act as a careful reviewer of your own work.
5
6 ---- TRAINING EXAMPLES (same as before) ----
7 {examples}
8
9 ---- YOUR FIRST-DRAFT ANALYSIS ----
10 DICTIONARY:
11 {first_draft_dictionary}
12
13 GRAMMAR RULES:
14 {first_draft_rules}
15
16 FIRST-DRAFT TRANSLATIONS:
17 {first_draft_translations}
18

```

```

19 --- YOUR TASK ---
20 1. CRITIQUE: For each translation above, check whether it is consistent with your
21 dictionary and grammar rules. List specific contradictions or uncertainties.
22 Be precise: quote the rule or entry that is violated or missing.
23
24 2. REVISED DICTIONARY: Update entries if needed.
25
26 3. REVISED GRAMMAR RULES: Update rules if needed.
27
28 4. FINAL TRANSLATIONS: Produce corrected translations based on your critique.
29 If a translation was already correct, keep it unchanged.
30
31 CRITICAL: Your task is to {translation_instruction}. Every line in TRANSLATIONS must be
32 in {target_language}.
33 Do NOT produce text in any other language.
34
35 Format your response EXACTLY as:
36 CRITIQUE:
37 [numbered list of issues, or "No contradictions found."]
38
39 DICTIONARY:
40 [entries]
41
42 GRAMMAR RULES:
43 [rules]
44
45 {TRANSLATIONS_BLOCK}

```

El recordatorio **CRITICAL** se añadió a posteriori, ya que tras observar que, en algunos experimentos, los modelos cambiaban de dirección durante la autocorrección y producía texto en la lengua desconocida en lugar de traducir al inglés. También se mantiene el bloque dinámico de traducciones para asegurar que el segundo turno devuelva exactamente el mismo número de respuestas que el primero. Este quizás es el prompt más complejo y, por desgracia, el menos certero. Parece ser que el modelo no llega a comprender su función de crítico y, por consiguiente, suele empeorar la hipótesis inicial en todos los casos.

5.3. Extracción automática de traducciones

Las respuestas del modelo no contienen únicamente traducciones. Normalmente incluyen también análisis morfológico, diccionario, reglas gramaticales y comentarios adicionales. Por ello, antes de calcular las métricas es necesario aislar únicamente las traducciones finales. Esta parte ha sido todo un reto, ya que primero había que averiguar la manera de pedirle al modelo el formato exacto de fichero que debe devolver y, posteriormente, rescatar las frases traducidas de forma automática para ahorrar tiempo, ya que la extracción manual sería poco práctica.

5.3.1. Algoritmo de extracción

El extractor sigue tres pasos principales:

1. **Localización de la sección.** Se busca la última aparición de **TRANSLATIONS:** en la respuesta, no la primera. Esta decisión es necesaria porque estrategias como M5 pueden mencionar la palabra *Translations* en la sección de crítica antes de llegar a la sección final de resultados.
2. **Extracción indexada por número.** Dentro de esa sección se analizan líneas del tipo **N. texto**. El número escrito por el modelo determina la posición de la hipótesis. Así, si el modelo escribe **3. texto**, ese texto se asigna siempre a la tercera frase de test, aunque las líneas anteriores no hayan sido reconocidas. Este cambio se ha tenido que implementar debido a un problema de extracciones anteriores, ya que cuando se perdía la primera traducción, las siguientes se desplazaban y acababan comparándose con referencias incorrectas, por lo que las métricas calculadas no eran fiables.
3. **Filtrado.** Los candidatos pasan por varios filtros para descartar entradas de diccionario, instrucciones de revisión y análisis metalingüístico. Por ejemplo, se eliminan líneas con patrones del tipo **palabra = significado**, instrucciones como **add** o **delete** acompañadas de términos como **rule** o **morpheme**, y explicaciones que contienen expresiones como **word order**, **suffix** o **allomorph**. Para ello lo que más ha sido de ayuda es la revisión manual de los logs crudos que devolvía el modelo, donde podía observar que filtrados son necesarios para la extracción de textos traducidos. No obstante, este enfoque tiene una limitación importante, y es que al depender de expresiones y formatos concretos, puede comportarse de forma distinta según el modelo utilizado y según cómo cada modelo organice su respuesta.

5.3.2. Gestión de respuestas incompletas

Cuando el extractor no encuentra una traducción para una posición, la deja como **None**. Durante el cálculo de métricas, esas posiciones se rellenan con cadenas vacías antes de calcular chrF++ a nivel de corpus. Así, una respuesta incompleta penaliza el resultado en lugar de evaluarse solo sobre las frases que el modelo sí respondió. Esta decisión se recoge también en la Tabla 5.2.

Los campos **missing_indices** y **complete** del JSON de métricas registran explícitamente qué posiciones se perdieron:

Código 5.18: Ejemplo de extracción incompleta en el JSON de métricas

```

1 "primary_metrics": {
2   "corpus_chrfpp": 23.4,
3   "complete": false,
4   "missing_indices": [0, 2], // posiciones 1 y 3 sin traducción
5   "num_evaluated": 4,
6   "num_expected": 6,
7   "hypotheses": [
8     "", // pos. 0 vacía — penaliza en el corpus
9     "Did he see us?",
10    "", // pos. 2 vacía — penaliza en el corpus
11    "Do we beat them?",
12    "Did we know you?",
13    "Did we go?"

```

```

14],
15"references": [
16"Do you(sg) sleep?",
17"Did he see us?",
18"Do they know you(pl)?",
19"Do we beat them?",
20"Did we know you(sg)?",
21"Did we go?"
22]
23}

```

5.3.3. Detección de bucles de repetición

Durante los experimentos se observó que algunos modelos, especialmente Llama-3.1-8B en prompts largos como M4 y M5, podían entrar en bucles de generación repetitiva. En esos casos, el modelo repetía entradas del diccionario hasta agotar el límite de tokens y no llegaba a producir la sección TRANSLATIONS. El código 5.19 muestra un ejemplo real de un problema encontrado.

Código 5.19: Fragmento de respuesta con bucle de repetición (Inuktitut, Llama-3.1-8B, M1)

```

1 DICTIONARY:
2 ...
3 197. The suffix "-ha" appears to indicate a third-person
4 singular subject performing an action.
5 198. The suffix "-ha" appears to indicate a third-person
6 singular subject performing an action.
7 199. The suffix "-ha" appears to indicate a third-person
8 singular subject...
9 [fin de tokens — nunca se genera TRANSLATIONS]

```

Como solución, se decidió que la función `call_model` comprobara si la respuesta contenía seis o más líneas idénticas consecutivas. Si se detectaba un bucle, la llamada se repetía con una temperatura ligeramente superior, con el objetivo de introducir más variación y romper la repetición. En la configuración base, las llamadas se realizaron con `temperature=0.0`, `top_p=1.0` y sin modificar `top_k`, manteniendo el valor por defecto de cada proveedor. Cuando se detectaba un bucle, se repetía la llamada con `temperature=0.4`, manteniendo el resto de parámetros sin cambios. Se permitieron hasta cuatro intentos.

5.4. Problemas encontrados y soluciones

5.4.1. Limitaciones de las APIs

Durante la ejecución de los experimentos se detectaron varias limitaciones derivadas del uso de APIs externas. La Tabla 5.1 resume las principales incidencias y las soluciones adoptadas. Aunque el mayor problema de todos siempre ha sido la escasa cuota de peticiones para cualquier modelo, que en muchas ocasiones cortaba a mitad de proceso y obligaba a reanudarlos varias horas después, o incluso al día siguiente, cuando se restablecía el límite de uso de la API.

Tabla 5.1: Principales incidencias relacionadas con las APIs utilizadas.

Incidencia	Consecuencia	Solución
Límite de tasa (429)	Peticiones rechazadas temporalmente.	Reintentos automáticos con esperas crecientes (30 s, 60 s y 90 s).
Límite diario de solicitudes	Experimentos truncados a mitad de ejecución.	Pipeline en dos fases: lo capturado permanece en <code>raw_logs/</code> aunque la sesión se interrumpa.
Presupuesto de tokens insuficiente	El modelo GPT-OSS-120B agotaba los tokens en el razonamiento interno antes de generar TRANSLATIONS.	Se elevó <code>max_tokens</code> a 8000 para este modelo (4096 para los demás).
Respuesta de contenido vacío	El campo <code>content</code> llegaba vacío en modelos razonadores; el sistema recurría al campo <code>reasoning</code> .	Se prioriza <code>content</code> ; si está vacío, se usa <code>reasoning</code> como respaldo.

5.4.2. Problemas de extracción y métricas

Además de las limitaciones de las APIs, también aparecieron problemas relacionados con la extracción automática de traducciones y el cálculo de métricas. Aunque ya hayan sido explicadas en el proceso de desarrollo, la tabla 5.2 permite visualizar y resume las principales decisiones técnicas adoptadas.

5.4.3. Otras limitaciones

Algunas limitaciones no pueden resolverse por completo mediante cambios en el código. La dependencia de proveedores externos implica que el comportamiento de los modelos puede variar entre sesiones. Además, el control M0 solo permite detectar indicios de conocimiento previo, pero no demostrar con certeza que una lengua esté ausente de los datos de preentrenamiento.

También es importante tener en cuenta que las métricas automáticas no sustituyen al análisis manual. Dos traducciones formalmente distintas pueden ser lingüísticamente equivalentes, y una respuesta con chrF++ elevado puede contener un error gramatical relevante.

Tabla 5.2: Principales decisiones técnicas adoptadas durante el desarrollo.

Problema	Efecto observado	Decisión adoptada
Desplazamiento posicional del extractor	Si se perdía la traducción 1, las traducciones 2, 3... se emparejaban con referencias incorrectas, inflando el chrF++.	Extracción indexada por el número que escribe el modelo (Sección 5.3.1).
Cabecera TRANSLATIONS ambigua	M5 menciona «Translation 1 (...)» en la sección de crítica antes de la sección real; el extractor anclaba en la primera aparición.	Se usa la <i>última</i> aparición de TRANSLATIONS: con dos puntos obligatorios.
Marcador === END === en la última línea	El modelo a veces escribía el marcador pegado a la última traducción (...ate the bread.=== END ===).	La función <code>clean()</code> elimina el sufijo <code>=== END ===</code> de cualquier candidato.
Evaluación parcial de respuestas incompletas	Evaluar solo las traducciones extraídas favorecía artificialmente las salidas incompletas.	Relleno con cadenas vacías y penalización en el cálculo de corpus.
Bucles de repetición del modelo	Algunos modelos agotaban los tokens repitiendo entradas del diccionario sin llegar a generar traducciones.	Detección automática (<code>_has_repetition</code>) y rein-tento con <code>temperature=0.4</code> (Sección 5.3.3).

6. Resultados

Este capítulo presenta los resultados obtenidos sobre los 11 puzles, que se indicarán en las tablas de los resultados, de dirección lengua $\rightarrow$ inglés del benchmark LINGUINI, organizados por metodología. La métrica principal utilizada y representada en las tablas es **chrF++**, calculada a nivel de corpus sobre el conjunto de frases de test de cada puzle. Esta métrica se toma como referencia principal porque permite captar coincidencias parciales entre la hipótesis generada y la referencia, algo especialmente útil en frases cortas y en problemas donde pequeños cambios morfológicos pueden afectar de forma notable a la puntuación.

Los modelos evaluados son **Llama-3.1-8B**, **Llama-3.3-70B** y **GPT-OSS-120B**. Como se indicó en la Tabla 4.2, en este trabajo se agrupan en tres niveles orientativos de capacidad: débil, medio y fuerte. Esta clasificación no corresponde a una categoría oficial de los proveedores, sino más bien se trata de una división interna que basamos en el tamaño del modelo. Así, Llama-3.1-8B se utiliza como modelo de menor capacidad, Llama-3.3-70B como modelo intermedio y GPT-OSS-120B como modelo de mayor capacidad. Los resultados se han recalculado a partir de los logs crudos generados por el sistema experimental, separando la captura de respuestas del cálculo posterior de métricas, como se describe en la sección 5.1.

6.1. M0 — Verificación previa

Tabla 6.1: chrF++ de M0 (verificación previa). Un valor inferior a 20 indica que el modelo no reconoce la lengua según nuestro criterio. En negrita, valor más alto por puzle.

Lengua	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	1.6	1.0	6.5
Ngemba	4.4	4.1	9.9
Yonggom	5.1	3.8	6.2
Warlpiri	4.7	4.2	9.1
Apurinã	0.2	3.7	7.5
Marind costero	1.4	2.0	4.0
Dyirbal	7.0	7.9	17.9
Nubio kunuz	4.5	4.6	10.0
Inuktitut	3.1	3.3	8.8
Tzeltal	3.9	4.7	5.0
Lakota	8.4	6.5	14.3
Media	4.0	4.2	9.0

M0 actúa como control exploratorio previo de verificación de contaminación de datos. Su objetivo se reduce a comprobar si el modelo puede producir una traducción razonable sin

haber recibido ningún ejemplo de entrenamiento. Un chrF++ inferior a 20 se interpreta como indicio de que el modelo no reconoce la lengua de forma suficiente como para resolver la tarea directamente (Sánchez y cols., 2024).

Como se observa en la tabla 6.1, los tres modelos se mantienen por debajo de ese umbral en todos los puzzles. Las medias son bajas en los tres casos: 4.0 para Llama-3.1-8B, 4.2 para Llama-3.3-70B y 9.0 para GPT-OSS-120B. Esto no demuestra que las lenguas estén completamente ausentes de los datos de preentrenamiento, pero sí reduce la probabilidad de que los resultados posteriores se deban a un conocimiento directo de las respuestas. Además, al revisar las salidas de M0, se observa que en muchos casos el modelo responde con una hipótesis claramente aleatoria o con una frase explicativa indicando que no dispone de información suficiente para traducir la lengua.

Los valores más altos aparecen en GPT-OSS-120B para Dyirbal, con chrF++=17.9, y para Lakota, con chrF++=14.3. Aunque ambos siguen por debajo del umbral, son los casos más cercanos a 20 y, por tanto, conviene interpretarlos con cautela. En Dyirbal, además, los otros dos modelos también obtienen valores relativamente altos en comparación con el resto de lenguas, lo que podría indicar una mayor presencia pública de esta lengua o una mayor coincidencia superficial con la referencia. En Lakota, el valor elevado aparece principalmente en GPT-OSS-120B, por lo que puede deberse a una respuesta parcialmente próxima a la referencia sin que ello implique necesariamente conocimiento directo del puzzle.

En conjunto, M0 permite mantener el supuesto experimental básico: antes de recibir los ejemplos del puzzle, los modelos no parecen disponer de información suficiente para traducir correctamente las frases de test.

6.2. M1 — Baseline

Tabla 6.2: chrF++ de M1 (baseline: todos los ejemplos de entrenamiento de una sola vez). En negrita, valor más alto por puzzle.

Lengua	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	14.7	81.7	100.0
Ngemba	—	50.0	41.5
Yonggom	38.3	43.8	36.5
Warlpiri	44.6	73.1	36.8
Apurinã	12.3	33.5	27.0
Marind costero	40.4	37.7	28.6
Dyirbal	32.1	55.5	22.3
Nubio kunuz	47.4	47.7	31.3
Inuktitut	52.0	57.2	40.5
Tzeltal	33.6	48.7	38.9
Lakota	39.3	62.6	42.7
Media	35.5	53.8	40.6
(σ)	(± 12.4)	(± 13.8)	(± 19.8)

M1 establece la línea de base del experimento. En esta metodología, el modelo recibe todos los ejemplos de entrenamiento en un único prompt y debe inferir directamente el diccionario, las reglas gramaticales y las traducciones de test.

El mejor comportamiento medio en M1 lo obtiene Llama-3.3-70B, con chrF++=53.8. Este modelo supera a los otros dos en la mayoría de puzzles y destaca especialmente en Hankun (81.7) y Warlpiri (73.1). Lo más relevante no es solo el valor absoluto, sino el incremento respecto a M0: este modelo pasa de una media de 4.2 en la verificación previa a 53.8 en M1, lo que supone una mejora de 49.6 puntos. Este salto indica que el modelo no resolvía los puzzles de forma directa antes de recibir ejemplos, pero sí es capaz de aprovechar el contexto de entrenamiento cuando se le proporciona en el prompt.

GPT-OSS-120B obtiene una media de 40.6. Su resultado más alto aparece en Hankun (100.0), donde supera claramente al resto de modelos, pero en otros puzzles queda por debajo del modelo de 70B. Los casos más bajos son Dyirbal (22.3), Apurinã (27.0) y Marind costero (28.6). Respecto a M0, GPT-OSS-120B pasa de una media de 9.0 a 40.6, con una mejora de 31.6 puntos. Esto indica que, aunque el modelo grande puede resolver muy bien algunos puzzles concretos, su rendimiento no es uniforme, por ello también calculamos la desviación típica que es ligeramente más elevada en el modelo de razonamiento frente a los otros dos.

Llama-3.1-8B alcanza una media de 35.5 sobre los puzzles con datos disponibles. Aunque es el modelo más pequeño, consigue resultados competitivos en varios casos, como Inuktitut (52.0), Nubio kunuz (47.4), Warlpiri (44.6) y Marind costero (40.4). Frente a M0, donde obtenía una media de 4.0, M1 supone una mejora de 31.5 puntos. El caso de Ngemba queda sin valor extraíble porque Llama-3.1-8B agotó el presupuesto de tokens durante la fase de análisis del diccionario, antes de alcanzar la sección **TRANSLATIONS**. Esto puede suceder debido a que Ngemba es uno de los puzzles con más pares de entrenamiento de todo el benchmark, lo que provoca que el modelo más pequeño genere un análisis excesivamente extenso antes de producir las traducciones.

La conclusión principal de M1 es que proporcionar todos los ejemplos de entrenamiento en el prompt produce una mejora clara respecto al control M0 en los tres modelos. Sin embargo, el tamaño del modelo no explica por sí solo el rendimiento. Llama-3.3-70B es claramente el mejor baseline, mientras que GPT-OSS-120B solo domina en algunos casos concretos. Esto refuerza la necesidad de comparar metodologías y no asumir que el modelo de mayor tamaño será automáticamente el más eficaz en todos los puzzles.

6.3. M2 — Incremental acumulativo

Tabla 6.3: chrF++ de M2 (incremental acumulativo, último paso). Se reporta el último paso, cuando el modelo ha recibido todos los ejemplos, como resultado principal (véase tabla 5.2). En negrita, valor más alto por puzle.

Lengua	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	54.8	79.3	91.7
Ngemba	21.6	46.9	42.3
Yonggom	38.7	32.0	36.2
Warlpiri	32.3	53.0	40.8
Apurinã	23.5	35.5	55.1
Marind costero	24.1	26.5	19.6
Dyirbal	33.9	38.1	35.9
Nubio kunuz	33.4	39.5	41.9
Inuktitut	38.0	50.3	65.9
Tzeltal	39.5	41.9	43.9
Lakota	29.4	53.1	60.1
Media	33.6	45.1	48.5
(σ)	(± 9.0)	(± 13.6)	(± 18.2)

M2 es una de las metodologías principales del trabajo. A diferencia de M1, no presenta todos los ejemplos de entrenamiento de una sola vez, sino que incorpora subconjuntos crecientes y obliga al modelo a revisar el diccionario y las reglas acumuladas en cada paso. Para evitar una selección optimista, se reporta siempre el último paso, es decir, aquel en el que el modelo ya ha recibido todos los ejemplos.

El comportamiento de M2 depende mucho del modelo. En GPT-OSS-120B, M2 mejora claramente respecto a M1: la media pasa de 40.6 a 48.5. Las mejoras más importantes aparecen en Apurinã (+28.1), Inuktitut (+25.4) y Lakota (+17.4). En estos casos, la presentación progresiva parece ayudar al modelo a construir hipótesis más estables antes de llegar a la traducción final.

Sin embargo, como podemos observar, M2 no mejora todos los puzles respecto a M1. En GPT-OSS-120B baja en Hankun (-8.3), Marind costero (-9.0) y Yonggom (-0.3), y apenas mejora Ngemba (+0.8). Esto muestra que la acumulación incremental puede introducir ruido cuando el modelo arrastra hipótesis incorrectas de pasos anteriores o cuando el baseline ya resuelve bien el puzle.

En Llama-3.1-8B ocurre lo contrario: M2 obtiene una media de 33.6, inferior a su M1 (35.5). Aun así, hay mejoras concretas en Hankun (de 14.7 a 54.8), Apurinã (de 12.3 a 23.5), Dyirbal (de 32.1 a 33.9) y Tzeltal (de 33.6 a 39.5). El patrón sugiere que el modelo pequeño puede beneficiarse de la estrategia incremental en algunos puzles, pero le cuesta mantener de forma estable el diccionario y las reglas acumuladas a lo largo de muchos pasos.

En Llama-3.3-70B, M2 alcanza una media de 45.1, por debajo de su baseline M1 (53.8). Solo mejora en Apurinã y no con diferencia relevante (de 33.5 a 35.5), mientras que empeora en la mayoría de los demás puzles. Esto indica que, para el 70B, la presentación directa de todos los ejemplos resulta más eficaz que el proceso incremental.

En conjunto, M2 es especialmente útil para GPT-OSS-120B, pero no se comporta igual en los otros modelos. La estrategia parece requerir una capacidad suficiente para mantener y revisar hipótesis acumuladas sin degradarlas; cuando esto no ocurre, el proceso incremental puede ser peor que presentar todo el contexto desde el principio.

6.4. M3 — Step-by-step lingüístico

Tabla 6.4: chrF++ de M3 (step-by-step lingüístico, etapa de sintaxis). En negrita, valor más alto por puzle. * = extracción incompleta en alguna etapa.

Lengua	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	9.3	17.4	25.8
Ngemba	14.2	19.8	13.2
Yonggom	19.8	15.6	9.0
Warlpiri	14.3	11.9	16.8
Apurinã	20.4	20.9	6.0
Marind costero	10.8	10.2	4.6
Dyirbal	31.8	28.9	28.9
Nubio kunuz	20.7	15.8	17.1
Inuktitut	18.4	17.9	10.7
Tzeltal	19.5	10.9*	10.6
Lakota	34.8*	37.9	37.8
Media	19.5	18.8	16.4
(σ)	(± 7.5)	(± 7.9)	(± 9.9)

M3 es la metodología con peor rendimiento global. Las medias son 19.5 para Llama-3.1-8B, 18.8 para Llama-3.3-70B y 16.4 para GPT-OSS-120B. En ningún modelo supera a M1 ni a M2 como estrategia general. La explicación más probable está en la partición heurística de los ejemplos. M3 divide el entrenamiento en cuatro etapas —léxico, fonología, morfosintaxis y sintaxis—, pero los puzles de Linguini no están diseñados ni ordenados para que cada bloque ilustre de forma clara uno de esos niveles. Como consecuencia, el modelo puede recibir en una etapa ejemplos que no corresponden realmente al fenómeno que se le está pidiendo analizar.

Aun así, M3 deja un resultado interesante: el 8B obtiene la mejor media de los tres modelos en esta metodología y supera a GPT-OSS-120B en varios puzles. Esto puede deberse a que la división en etapas reduce el tamaño del contexto que el modelo debe procesar en cada turno. Para un modelo pequeño, esa simplificación puede ser útil, aunque no lo suficiente como para superar a metodologías más directas como M1.

La conclusión es que el razonamiento paso a paso no debe descartarse, aunque esta metodología concreta no parece ser la más adecuada para el benchmark utilizado. Para funcionar mejor, necesitaría una partición lingüística más cuidada. Si los ejemplos no están seleccionados manualmente para ilustrar cada fenómeno, la estructura por etapas puede introducir más ruido que ayuda.

6.5. M4 — Inspiración humana

Tabla 6.5: chrF++ de M4 (inspiración humana). Hankun no se evalúa: es el puzle cuyo razonamiento humano se emplea como guía, para evitar *data leakage*. En negrita, valor más alto por puzle. —: no extraíble.

Lengua	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	—	—	—
Ngemba	24.4	46.5	40.4
Yonggom	—	33.6	43.0
Warlpiri	—	49.3	39.5
Apurinã	45.9	38.6	46.6
Marind costero	22.8	28.5	27.1
Dyirbal	—	45.7	17.9
Nubio kunuz	46.7	51.3	39.1
Inuktitut	—	51.1	44.9
Tzeltal	33.1	51.0	46.8
Lakota	31.4	65.5	63.3
Media[‡]	34.0	46.2	40.9
(σ)	(± 9.4)	(± 9.9)	(± 11.4)

M4 utiliza como guía una resolución humana del puzle de Hankun. El modelo que mejor aprovecha esta estrategia es Llama-3.3-70B, con una media de 46.2. Supera a GPT-OSS-120B en la mayoría de puzles evaluados bajo M4 y obtiene resultados especialmente altos en Lakota (65.5), Nubio kunuz (51.3), Inuktitut (51.1), Tzeltal (51.0) y Ngemba (46.5). Esto sugiere que el 70B es capaz de abstraer bien el procedimiento mostrado en el ejemplo humano y aplicarlo a nuevas lenguas.

GPT-OSS-120B obtiene una media de 40.9. Sus mejores resultados aparecen en Lakota (63.3), Tzeltal (46.8), Apurinã (46.6), Inuktitut (44.9) y Yonggom (43.0). En varios de estos casos mejora claramente respecto a M1, lo que indica que la guía humana puede ayudar al modelo a organizar mejor el análisis cuando el puzle requiere prestar atención a marcas morfológicas o regularidades poco evidentes.

Llama-3.1-8B obtiene una media de 34.0, pero debe interpretarse con cautela porque cuatro puzles no pudieron completarse o no produjeron traducciones extraíbles, por los problemas de extracción automática y agotamiento de tokens descritos en las secciones 5.3.2 y 5.3.3. El prompt de M4 es el más largo del sistema, ya que incluye el razonamiento humano completo, y esto afecta especialmente al modelo pequeño. En algunos casos el 8B genera análisis demasiado extensos y agota el presupuesto de tokens antes de llegar a la sección de traducciones.

El resultado general de M4 se podría decir que es positivo, aunque no uniforme. La inspiración humana no actúa como una solución universal, pero sí mejora algunos puzles y resulta especialmente útil para Llama-3.3-70B. Esto sugiere que los ejemplos de razonamiento humano pueden ser una línea prometedora, siempre que se controle la longitud del prompt y se evite transferir contenido lingüístico del ejemplo al nuevo puzle. Tal vez se trate de darle al modelo más ejemplos de razonamiento para que funcione mejor.

6.5.1. Comparación cualitativa con la resolución humana

Además de los resultados cuantitativos, M4 permite analizar hasta qué punto una guía basada en razonamiento humano ayuda al modelo a organizar mejor su proceso de deducción. Para ello, se compara la resolución manual del puzle de Hankun con las respuestas generadas por los modelos, prestando atención no solo a la traducción final, sino también al diccionario, las reglas gramaticales y las hipótesis formuladas durante la resolución.

La comparación con la respuesta del modelo en M1 muestra un patrón claro: el modelo identifica parte del léxico básico, especialmente algunos verbos y pronombres, pero tiene más dificultades para aplicar de forma consistente las marcas gramaticales. En el caso de Hankun, el sistema aspectual, es decir, la distinción entre presente y pasado, resulta especialmente problemático. El modelo llega a mencionar en su razonamiento la posibilidad de que ? esté relacionado con el pasado, pero no aplica esa hipótesis de manera uniforme a todas las formas verbales. Esto provoca traducciones en las que el significado léxico principal puede ser correcto, pero el tiempo verbal no coincide con la referencia. Este tipo de error es importante porque muestra que el problema no está solo en reconocer palabras aisladas, sino en mantener y aplicar una regla gramatical de forma sistemática.

En M2, los primeros pasos producen traducciones más débiles porque el modelo todavía dispone de poca evidencia para deducir el sistema pronominal y aspectual completo. A medida que recibe más ejemplos, el rendimiento suele mejorar significativamente, aunque en algunos casos las hipótesis acumuladas introducen ruido y afectan a pasos posteriores. Esta evolución contrasta con la resolución humana, donde cada nuevo ejemplo permitió refinar las hipótesis anteriores sin perder las regularidades ya identificadas.

Este contraste sugiere que una de las diferencias principales entre ambos procesos está en la estabilidad de las hipótesis. La persona que resuelve el puzle puede mantener una regla provisional, pero también revisarla cuando aparece nueva evidencia. El modelo, en cambio, puede formular una regla correcta y aun así no aplicarla de forma consistente, o puede conservar una hipótesis incorrecta durante varios pasos. Por ello, el análisis cualitativo resulta útil para complementar las métricas automáticas y entender mejor qué tipo de errores aparecen durante la resolución.

6.6. M5 — Segundo revisor

Tabla 6.6: chrF++ de M5 (segundo revisor, autocorrección iterativa). Se reporta el turno 2 (revisión final) como resultado principal. En negrita, T2 más alto por puzzle entre los modelos disponibles. —: pendiente o no extraíble. † = T1 vacío: el modelo agotó tokens antes de producir traducciones.

Lengua	8B T1	8B T2	70B T1	70B T2	GPT T1	GPT T2
Hankun	10.7	8.3	77.8	26.6	72.0	3.3
Ngemba	39.2	32.9	49.4	20.9	† 0.0	5.8
Yonggom	36.1	36.0	42.8	38.2	31.4	40.2
Warlpiri	44.6	40.9	58.2	44.1	† 0.0	12.2
Apurinã	20.1	24.4	50.6	27.4	42.9	38.2
Marind costero	† 0.0	29.3	47.2	47.3	13.2	16.1
Dyirbal	32.1	32.1	44.2	28.3	† 0.0	31.4
Nubio kunuz	47.4	47.9	47.7	16.6	† 0.0	22.4
Inuktitut	41.5	41.5	53.2	32.3	43.4	43.4
Tzeltal	33.6	31.7	42.7	45.5	59.0	59.0
Lakota	45.9	46.3	69.2	28.0	59.6	49.9
Media T2		33.9		32.4		29.3
(σ T2)		(± 10.6)		(± 9.8)		(± 17.8)

† indica que el turno 1 no llegó a producir ninguna traducción porque el modelo agotó tokens sin completar la sección TRANSLATIONS.

M5 evalúa si una segunda fase de revisión permite mejorar el primer borrador generado por el modelo. Para ello se comparan dos turnos: T1, que equivale a una primera solución, y T2, donde el modelo revisa su propia respuesta. Los resultados muestran que esta estrategia es inestable y que la autocorrección no garantiza una mejora.

En GPT-OSS-120B, la media final de T2 es 29.3, por debajo de M1 (40.6), M2 (48.5) y M4 (40.9). El modelo mejora en algunos casos, como Yonggom (31.4 a 40.2), Marind costero (13.2 a 16.1) y Dyirbal (0.0 a 31.4), y mantiene el resultado en Inuktitut y Tzeltal. Sin embargo, también hay caídas muy fuertes, especialmente en Hankun (72.0 a 3.3), Lakota (59.6 a 49.9) y Apurinã (42.9 a 38.2). En los casos marcados con †, el primer turno no produjo traducciones válidas, por lo que el segundo turno no puede interpretarse como una revisión real de un borrador completo.

En Llama-3.1-8B, la media de T2 es 33.9, ligeramente inferior a su M1 (35.5), pero superior a la T2 de GPT-OSS-120B. El comportamiento es más moderado: hay mejoras en Apurinã, Marind costero, Nubio kunuz y Lakota, estabilidad en Dyirbal e Inuktitut, y empeoramientos en Hankun, Ngemba, Yonggom, Warlpiri y Tzeltal. Esto sugiere que el segundo turno puede ayudar al modelo pequeño en algunos casos, pero no de forma suficientemente consistente.

En Llama-3.3-70B, la media de T2 es 32.4, muy por debajo de su baseline M1 (53.8). Este modelo muestra algunos de los casos más claros de sobrecorrección. Por ejemplo, Hankun baja de 77.8 a 26.6, Lakota de 69.2 a 28.0, Nubio kunuz de 47.7 a 16.6 e Inuktitut de 53.2 a 32.3. Solo se observan mejoras en Marind costero, donde el cambio es mínimo (47.2 a 47.3), y en Tzeltal (42.7 a 45.5).

La conclusión principal de M5 es que la autocorrección sin retroalimentación externa puede

ser peligrosa cuando el primer borrador ya es bueno. El modelo revisa sus propias hipótesis, pero esas hipótesis pueden estar mal formuladas o ser aplicadas de forma incorrecta. En esos casos, el segundo turno no corrige el error, sino que lo amplifica. Este comportamiento coincide con una limitación habitual de los enfoques tipo *Self-Refine*: si la crítica procede del mismo modelo y no hay una señal externa de validación, la revisión puede reforzar los fallos iniciales (Madaan y cols., 2023). Por falta de recursos, la prueba de una crítica externa queda fuera del alcance de esta entrega, aunque sería interesante estudiarla en un trabajo futuro.

6.7. Experimentos complementarios

Además del experimento principal, centrado en las metodologías M0–M5 sobre puzzles de traducción lengua $\rightarrow$ inglés, se realizaron varias pruebas adicionales. Estas pruebas no forman parte de la comparación principal, pero ayudan a entender mejor algunos aspectos del sistema, así como la importancia de seleccionar buenos ejemplos, la dificultad de traducir en la dirección inversa y el comportamiento de los modelos en tareas lingüísticas distintas de la traducción.

6.7.1. M2.2 — Exploración combinatoria por tamaños crecientes

La estrategia M2 utilizada en el experimento principal trabaja con prefijos crecientes del conjunto de entrenamiento. Es decir, en cada paso añade una nueva frase siguiendo el orden original del puzzle: primero las frases f_1 y f_2 , después f_1 , f_2 y f_3 , y así sucesivamente. Esta decisión permite mantener el número de llamadas bajo control, pero tiene una limitación evidente: asume que los primeros ejemplos son suficientemente informativos. Así, la variante M2.2 se planteó para estudiar precisamente ese punto. En lugar de evaluar un único subconjunto de tamaño k , esta estrategia prueba todas las combinaciones posibles de ejemplos para cada tamaño. De esta forma se puede observar hasta qué punto cambia el rendimiento del modelo dependiendo de qué frases concretas recibe al principio.

Diseño del experimento

Se ejecutó M2.2 sobre el puzzle de Hankun, en dirección hacia inglés, utilizando GPT-OSS-120B. El conjunto de entrenamiento contiene $n = 10$ frases y se limitó la exploración a $k_{\max} = 3$ para mantener el coste computacional dentro de un rango asumible. Esto da lugar a $\binom{10}{2} + \binom{10}{3} = 45 + 120 = 165$ combinaciones posibles.

El diccionario y las reglas gramaticales se acumulan a través de las iteraciones. Cada nueva combinación puede confirmar, refinar o contradecir la hipótesis vigente, del mismo modo que en M2. En total se completaron 162 de las 165 iteraciones previstas; las tres restantes fallaron por errores de red.

El primer resultado relevante es que la desviación típica baja mucho al pasar de $k = 2$ a $k = 3$. Con dos frases, su valor es $\sigma = 12.7$; con tres frases, baja a $\sigma = 4.8$. Esto indica que el aprendizaje inductivo del modelo se vuelve más estable cuando dispone de una mínima variedad de ejemplos. Con solo dos frases, la elección concreta del subconjunto puede cambiar por completo el resultado.

La mejor combinación de tamaño 2 está formada por las frases 7 y 10: nuʔrum kəmə ati lapkʰi kan ne y ati kəmə ŋa lapkʰi tʰɤ ne. Estas dos frases comparten el verbo lapkʰi (‘see’),

Tabla 6.7: Estadísticas de chrF++ en M2.2 por tamaño de combinación k (Hankun, GPT-OSS-120B). La columna «mejor combo» indica los índices de las frases de entrenamiento que forman la combinación con mayor chrF++.

k	$\binom{10}{k}$	μ	σ	máx	mín	mejor combo	peor combo
2	45	59.1	12.7	71.6	8.5	{7, 10}	{1, 2}
3	120	65.8	4.8	73.3	51.6	{4, 6, 10}	{1, 2, 5}

pero contrastan distintos pronombres y tiempos verbales. La primera introduce *nu rum* como segunda persona plural y una forma de presente, mientras que la segunda incluye *ati*, *ŋa* y una forma de pasado. Esa complementariedad explica que con solo dos ejemplos el modelo alcance chrF++=71.6.

En cambio, la peor pareja está formada por las frases 1 y 2: *ŋa ka kɔ ne* y *nɔ ʒip tu? ne*. Aunque ambas son frases simples, comparten poco vocabulario útil, usan verbos distintos y no muestran el marcador de transitividad *kəmə*. Con tan poca información, el modelo no puede reconstruir bien el sistema pronominal ni el sistema de tiempos, y el resultado cae a chrF++=8.5.

M2.2 no supera a M1 ni a M2 en este experimento. Esto es razonable, ya que M1 recibe los 10 ejemplos desde el primer turno y M2 acaba recibiendo todos en el último paso. La utilidad de M2.2 no está tanto en obtener la mejor puntuación final, sino en mostrar cuánto importa la selección inicial de ejemplos. En Hankun, la diferencia entre la mejor y la peor pareja es enorme: 71.6 frente a 8.5. Al añadir una tercera frase, el rango se reduce de forma clara, de 51.6 a 73.3. Esto sugiere que, en escenarios con presupuesto limitado de tokens o pocos ejemplos disponibles, elegir bien los ejemplos iniciales puede ser tan importante como el diseño del prompt.

Tabla 6.8: Comparación de M2.2 con M1 y M2 sobre Hankun (GPT-OSS-120B).

Metodología	chrF++
M1 — baseline (10 frases, 1 turno)	100.0
M2 — incremental (último paso)	91.7
M2.2 — mejor combinación $k = 3$	73.3
M2.2 — media $k = 3$ ($\sigma=4.8$)	65.8
M2.2 — media $k = 2$ ($\sigma=12.7$)	59.1
M2.2 — peor combinación $k = 2$	8.5

6.7.2. Dirección inversa: inglés → lengua desconocida

El experimento principal se centra en la dirección lengua → inglés, donde el modelo debe interpretar una lengua desconocida y producir una traducción en inglés. La dirección inversa, inglés → lengua desconocida, plantea una dificultad superior, ya que el modelo ya no solo tiene que reconocer patrones, sino generar formas en una lengua que no conoce.

Esta tarea obliga a producir morfemas, marcas funcionales y orden de palabras en la

lengua objetivo. Para explorar esta dirección se seleccionaron tres puzzles: Hankun, Yonggom y N|uuki. Hankun y Yonggom permiten comparar con los resultados obtenidos en la dirección lengua $\rightarrow$ inglés, mientras que N|uuki añade un caso con formas fonológicas especialmente alejadas del inglés.

Se ejecutó únicamente la estrategia M1, ya que el objetivo de esta prueba era obtener una primera comparación entre direcciones, no repetir todo el diseño experimental completo. Los resultados de M0 y M1 en dirección inversa se muestran en la tabla 6.9. Además, la tabla 6.10 compara directamente los resultados de M1 en ambas direcciones para los dos puzzles que cuentan con datos comparables.

Tabla 6.9: M0 y M1 en dirección inglés $\rightarrow$ lengua desconocida. * = el modelo entró en bucle de repetición antes de producir traducciones.

Lengua	Familia	M	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	Sino-tibetana	M0	1.4	1.5	2.0
		M1	0.0*	58.9	41.6
Yonggom	Trans-Nueva Guinea	M0	3.2	3.0	3.3
		M1	0.0*	21.6	18.1
N uuki	Tuu (chasquidos)	M0	0.2	2.2	3.6
		M1	0.0*	27.3	34.2

Tabla 6.10: Comparación lengua $\rightarrow$ inglés frente a inglés $\rightarrow$ lengua en M1 para los dos puzzles con datos en ambas direcciones. Δ negativo indica degradación al invertir la dirección.

Lengua	Dirección	Llama-3.1-8B	Llama-3.3-70B	GPT-OSS-120B
Hankun	lengua $\rightarrow$ inglés	14.7	81.7	100.0
	inglés $\rightarrow$ lengua	0.0*	58.9	41.6
	Δ	-14.7	-22.8	-58.4
Yonggom	lengua $\rightarrow$ inglés	38.3	43.8	36.5
	inglés $\rightarrow$ lengua	0.0*	21.6	18.1
	Δ	-38.3	-22.2	-18.4

El resultado más claro es que la dirección inversa es más difícil. Como se observa en la tabla 6.10, en Hankun y Yonggom todos los modelos empeoran al pasar de lengua $\rightarrow$ inglés a inglés $\rightarrow$ lengua. La caída es especialmente fuerte en GPT-OSS-120B para Hankun, donde pasa de 100.0 a 41.6. En Llama-3.3-70B también hay degradación, aunque conserva una parte mayor del rendimiento, con 58.9 en Hankun y 21.6 en Yonggom.

Llama-3.1-8B no produce ninguna traducción válida en los tres puzzles de dirección inversa. En todos los casos entra en bucle de generación repetitiva antes de llegar a la sección TRANSLATIONS. Este comportamiento coincide con lo observado en otras pruebas con prompts largos o tareas generativas complejas. El resultado sugiere que el modelo pequeño puede reconocer algunos patrones en dirección lengua $\rightarrow$ inglés, pero tiene muchas más dificultades cuando debe producir formas nuevas en una lengua desconocida.

Entre los modelos que sí generan respuestas, Llama-3.3-70B supera a GPT-OSS-120B en Hankun y Yonggom, mientras que GPT-OSS-120B obtiene mejor resultado en N|uuki. Este último caso es interesante porque N|uuki incluye consonantes de chasquido, como ꞥ, Ꞩ, ꞩ y Ɦ, que no tienen equivalente directo en inglés y pueden dificultar la producción exacta de formas. Aun así, GPT-OSS-120B logra el mejor resultado en ese puzle, como se muestra en la tabla 6.9.

En conjunto, esta prueba refuerza una idea importante: interpretar una lengua desconocida y traducirla al inglés no es equivalente a generar esa lengua. La dirección inversa exige una producción morfológica y fonológica mucho más precisa, y por tanto constituye una ampliación natural del trabajo, pero también más exigente. Este fenómeno podemos comprenderlo perfectamente comparándolo con el razonamiento que seguiríamos las personas. Si un individuo quisiera traducir una frase de su idioma natal, por ejemplo, el español, a una lengua que desconoce, como el italiano, sería más simple averiguar la traducción del italiano al español, ya que podemos reconocer patrones e incluso comprender algunas palabras por su similitud. Sin embargo, si quisiésemos traducir del español al italiano, nos encontraríamos con una situación diferente por falta de conocimiento de las palabras en italiano.

6.7.3. Tareas morfológicas y numéricas

El benchmark LINGUINI incluye, además de puzles de traducción, tareas que evalúan otras capacidades metalingüísticas. En esta prueba se seleccionaron tres tipos: *fill_blanks*, donde el modelo debe completar formas de un paradigma morfológico; *text_to_num*, donde debe convertir expresiones numéricas en dígitos; y *num_to_text*, donde debe producir expresiones numéricas en una lengua desconocida. Estas tareas no se incluyeron en el experimento principal porque requieren criterios de evaluación algo distintos. Aun así, resultan útiles para comprobar si las estrategias basadas en ejemplos también funcionan fuera de la traducción directa.

Importante mencionar que para estas pruebas complementarias no se reutilizó literalmente el prompt de traducción M1, sino una variante baseline adaptada al tipo de tarea. Más concretamente, en todos los casos se mantuvo la instrucción metalingüística general, pero se modificó la consigna final para ajustarla al formato esperado. Por ejemplo, en *fill_blanks*, el modelo debía inferir la transformación morfológica entre dos formas verbales y producir únicamente la segunda forma, o en el caso de los problemas de números, en *text_to_num*, debía inferir el sistema numérico y devolver solo el valor en dígitos, y en *num_to_text*, debía producir únicamente la expresión numérica en la lengua desconocida.

Guazacapán Xinka — completar paradigma verbal

El puzle de Guazacapán Xinka plantea una tarea de tipo *fill_blanks*. A partir de pares de formas verbales y sus significados en inglés, el modelo debe inferir una segunda forma verbal. Los ejemplos muestran alternancias como *piriy'* → *imbir'i* ('see') o *aplay'* → *inapal'a* ('open'), donde aparecen prefijos y cambios internos de la raíz. Los resultados obtenidos para esta tarea se muestran en la Tabla 6.11.

Los tres modelos identifican parte del patrón, especialmente la presencia del prefijo *in-*. Sin embargo, ninguno consigue generalizar correctamente todas las alternancias internas. El

Tabla 6.11: Resultados en Guazacapán Xinka (*fill_blanks*, M1 baseline). EM = Exact Match. La única forma correcta para todos los modelos es la cuarta (*her'oy* → *inher'o*).

Modelo	chrF++	EM	Patrón identificado
Llama-3.1-8B	36.4	1/4	Prefijo n- correcto, sufijos incorrectos
Llama-3.3-70B	44.6	1/4	Prefijo parcial, sufijo -ti erróneo
GPT-OSS-120B	47.8	1/4	Más cercano en sufijos, orden vocálico incorrecto

Exact Match es 1/4 en los tres casos, lo que indica que las formas generadas se parecen parcialmente a las referencias, pero no alcanzan la precisión formal necesaria.

Este resultado muestra una dificultad distinta a la traducción. En un puzzle de traducción, una respuesta puede ser parcialmente correcta aunque no coincida exactamente con la referencia. En cambio, en una tarea morfológica, pequeños cambios en una consonante, una vocal o un sufijo pueden hacer que la forma sea incorrecta. Por eso, aunque chrF++ indique cierta cercanía, el EM sigue siendo bajo.

Supyire — sistema numérico

El puzzle de Supyire evalúa la capacidad de inferir un sistema numérico a partir de pocos ejemplos. Se consideran dos direcciones: *text_to_num*, donde el modelo debe convertir expresiones en Supyire a números, y *num_to_text*, donde debe producir expresiones numéricas en Supyire.

Tabla 6.12: Resultados en Supyire (M1 baseline). EM = Exact Match. chrF++ se reporta para comparabilidad.

Modelo	text_to_num (n=2)		num_to_text (n=5)	
	chrF++	EM	chrF++	EM
Llama-3.1-8B	5.8	0/2	0.7	0/5
Llama-3.3-70B	9.3	0/2	46.6	0/5
GPT-OSS-120B	21.3	0/2	38.4	0/5

Ningún modelo alcanza Exact Match en las tareas numéricas. En *text_to_num*, GPT-OSS-120B obtiene el chrF++ más alto, aunque todavía lejos de una decodificación exacta. Esto sugiere que detecta parte de la estructura del sistema, pero no logra descomponer correctamente las expresiones numéricas.

En *num_to_text*, Llama-3.3-70B obtiene el mejor chrF++ con 46.6. Esto indica que genera formas parcialmente parecidas a las referencias, aunque ninguna coincide exactamente. Llama-3.1-8B fracasa especialmente en la dirección productiva, con chrF++=0.7, lo que refuerza el patrón observado en la dirección inglés → lengua: el modelo pequeño tiene muchas dificultades para producir formas en una lengua desconocida.

En conclusión, las tareas morfológicas y numéricas amplían la interpretación del expe-

rimento principal. Por un lado, muestran que los modelos pueden identificar regularidades parciales, como prefijos o morfemas numéricos, pero eso no siempre se traduce en respuestas exactas. Por otro lado, confirman que las tareas de producción son especialmente delicadas, y es que generar una forma nueva exige mayor precisión que reconocer una traducción aproximada. En conjunto, estas pruebas complementarias refuerzan una de las ideas centrales del trabajo: los modelos de lenguaje pueden extraer patrones lingüísticos a partir de pocos ejemplos, pero su razonamiento no siempre es estable ni suficientemente preciso cuando la tarea exige producir formas exactas en una lengua desconocida.

6.8. Resumen y análisis

La Tabla 6.13 recoge el mejor chrF++ obtenido por cada modelo y puzzle considerando las metodologías M1–M5. Esta tabla permite observar qué estrategia funciona mejor en cada caso concreto, aunque no debe interpretarse como que exista una metodología universalmente superior para todas las lenguas.

Tabla 6.13: Mejor chrF++ por puzzle (máximo sobre M1–M5) y metodología en que se obtuvo. En negrita, mejor modelo por puzzle entre los disponibles.

Lengua	8B	M _{8B}	70B	M _{70B}	GPT	M _{GPT}
Hankun	54.8	M2	81.7	M1	100.0	M1
Ngemba	32.9	M5	50.0	M1	42.3	M2
Yonggom	38.7	M2	43.8	M1	43.0	M4
Warlpiri	44.6	M1	73.1	M1	40.8	M2
Apurinã	45.9	M4	38.6	M4	55.1	M2
Marind costero	40.4	M1	47.3	M5	28.6	M1
Dyirbal	33.9	M2	55.5	M1	35.9	M2
Nubio kunuz	47.9	M5	51.3	M4	41.9	M2
Inuktitut	52.0	M1	57.2	M1	65.9	M2
Tzeltal	39.5	M2	51.0	M4	59.0	M5
Lakota	46.3	M5	65.5	M4	63.3	M4
Media	43.4		55.9		52.3	
(σ)	(± 6.7)		(± 12.3)		(± 18.8)	

Ranking de metodologías en GPT-OSS-120B

En GPT-OSS-120B, M2 es la metodología más fuerte. Obtiene el mejor resultado en 6 de los 11 puzzles y alcanza la mayor media entre metodologías, con chrF++=48.5. Este patrón indica que GPT-OSS-120B se beneficia especialmente de la presentación incremental acumulativa. El modelo parece aprovechar mejor una secuencia de hipótesis revisables que una única presentación completa de todos los ejemplos. Aun así, M2 no gana en todos los casos: M1 sigue siendo mejor en Hankun y Marind costero, M4 destaca en Yonggom y Lakota, y M5 obtiene el mejor resultado en Tzeltal.

Ranking de metodologías en Llama-3.1-8B

En Llama-3.1-8B el patrón es más repartido. M2 gana en Hankun, Yonggom, Dyirbal y Tzeltal; M1 gana en Warlpiri, Marind costero e Inuktitut; M5 gana en Ngemba, Nubio kunuz y Lakota; y M4 gana en Apurinã. M3 no gana en ningún puzle, y tampoco en los otros modelos, como podemos observar. Si se observan las medias, M1 es la metodología más estable para el 8B, con $\text{chrF}++=35.5$. M4 (34.0) y M5 (33.9) quedan muy cerca, mientras que M2 baja a 33.6. Esto sugiere que el modelo pequeño no siempre puede aprovechar bien la acumulación incremental de hipótesis. En su caso, presentar todos los ejemplos desde el principio o utilizar una guía humana puede ser igual o más eficaz que forzar una revisión paso a paso.

Ranking de metodologías en Llama-3.3-70B

Llama-3.3-70B muestra el patrón más claro a favor de M1. El baseline obtiene la mayor media, con $\text{chrF}++=53.8$, y es la metodología ganadora en Hankun, Ngemba, Yonggom, Warlpiri, Dyirbal e Inuktitut. M4 aparece como segunda estrategia más útil, con una media de 46.2, y gana en Apurinã, Nubio kunuz, Tzeltal y Lakota. M5 solo obtiene el mejor resultado en Marind costero, y M2 no gana en ningún puzle. A diferencia de GPT-OSS-120B, el 70B no se beneficia especialmente de M2. Para este modelo, disponer de todos los ejemplos desde el principio suele ser más eficaz que introducirlos de forma incremental. La autocorrección M5 también empeora claramente respecto a M1, lo que confirma que el segundo turno tiende a sobre corregir cuando el primer borrador ya es bueno.

Comparación entre modelos

La comparación entre modelos muestra que Llama-3.3-70B es el sistema más fuerte en este conjunto de experimentos. Obtiene la mejor media de resultados máximos por puzle, con $\text{chrF}++=55.9$, frente a 52.3 de GPT-OSS-120B y 43.4 de Llama-3.1-8B.

Este resultado es interesante porque rompe la expectativa simple de que el modelo con más parámetros debe obtener siempre mejores resultados. En este benchmark, el modelo de 70B supera con frecuencia a GPT-OSS-120B. Una posible explicación es que el rendimiento no depende solo del tamaño, sino también del ajuste instructivo del modelo, del proveedor, del formato de salida y de cómo cada arquitectura gestiona tareas de razonamiento con pocos ejemplos.

Llama-3.1-8B queda por debajo en media, pero no es irrelevante. En varios puzles obtiene resultados razonables y, en algunos casos, se acerca a los modelos mayores. Esto sugiere que parte del razonamiento necesario para estos puzles no requiere necesariamente modelos enormes, aunque los modelos más grandes suelen ser más robustos cuando la morfología o la estructura del puzle se complican.

Conclusión transversal sobre las metodologías

Los resultados no muestran una metodología ganadora universal. La mejor estrategia depende del modelo y del puzle. Para GPT-OSS-120B, M2 es la opción más prometedora; para Llama-3.3-70B, el baseline M1 sigue siendo el más fuerte; y para Llama-3.1-8B el comportamiento está algo más repartido.

La conclusión principal es que la forma de presentar los ejemplos importa, pero no de la misma manera para todos los modelos. Una estrategia incremental puede ayudar a un modelo capaz de mantener y revisar hipótesis, pero puede perjudicar a otro si arrastra errores entre pasos. Una guía humana puede orientar el análisis, pero también aumenta mucho la longitud del prompt. La autocorrección puede detectar algunos fallos, pero sin retroalimentación externa tiende a sobre corregir. Por tanto, el resultado más relevante del capítulo no es solo qué modelo obtiene la puntuación más alta, sino cómo cambia el comportamiento de cada modelo según la metodología. Esto confirma que los puzzles de olimpiadas lingüísticas son útiles no solo para medir rendimiento final, sino también para estudiar la estabilidad del razonamiento metalingüístico generado por los modelos.

7. Conclusiones

El objetivo principal de este trabajo de fin de grado era estudiar si los modelos de lenguaje de gran escala pueden resolver problemas de olimpiadas lingüísticas mediante razonamiento inductivo, sin recibir información explícita previa sobre la lengua analizada. Para ello se ha diseñado, implementado y evaluado un sistema experimental basado en distintas estrategias de *prompting*, aplicado sobre 11 puzzles de traducción del benchmark LINGUINI, en dirección lengua desconocida $\rightarrow$ inglés, utilizando tres modelos de distinto tamaño: Llama-3.1-8B, Llama-3.3-70B y GPT-OSS-120B. El trabajo parte del baseline directo comparable con el planteamiento de Sánchez y cols. (2024), pero incorpora estrategias adicionales orientadas a guiar el razonamiento metalingüístico del modelo mediante diccionarios, reglas revisables, razonamiento humano y autocorrección.

El primer resultado relevante es que la estrategia M2, basada en una presentación incremental acumulativa de los ejemplos, mejora claramente el baseline M1 en el modelo de razonamiento GPT-OSS-120B. En este modelo, M2 obtiene el mejor resultado en 6 de los 11 puzzles evaluados y alcanza una media de $\text{chrF}++=48.5$, frente a 40.6 en M1. Este comportamiento confirma parcialmente la hipótesis central del trabajo: presentar los ejemplos de forma progresiva y obligar al modelo a revisar su diccionario y sus reglas puede mejorar el razonamiento inductivo frente a una presentación completa de una sola vez. Sin embargo, el beneficio no es uniforme. En Llama-3.1-8B, M2 baja ligeramente respecto a M1, y en Llama-3.3-70B también queda por debajo del baseline. Esto indica que la estrategia incremental no es universalmente mejor, sino que requiere que el modelo pueda mantener hipótesis acumuladas, revisar contradicciones y no degradar el conocimiento construido en pasos anteriores.

El segundo hallazgo, quizá el más sorprendente, es que el modelo con mejor rendimiento global no es el de mayor tamaño. Llama-3.3-70B obtiene la mejor media de resultados máximos por puzzle, con $\text{chrF}++=55.9$, frente a 52.3 de GPT-OSS-120B y 43.4 de Llama-3.1-8B. Además, consigue el mejor resultado en 7 de los 11 puzzles. Este resultado invierte la jerarquía esperada por tamaño y sugiere que la capacidad de razonamiento inductivo lingüístico no escala de forma lineal con el número de parámetros, al menos dentro de los modelos y proveedores evaluados. Otros factores como el ajuste instructivo, el formato de salida, la estabilidad de generación o la gestión del contexto parecen tener un peso importante, al menos en las pruebas realizadas hasta ahora.

El tercer resultado importante se observa en M5, la estrategia de autocorrección iterativa. Aunque la hipótesis inicial era que una segunda fase de revisión podría corregir contradicciones internas, los resultados muestran que la autocorrección sin retroalimentación externa tiende a empeorar el rendimiento en la mayoría de casos. El efecto es especialmente claro en Llama-3.3-70B, donde algunos primeros borradores muy buenos se degradan tras la revisión: Hankun baja de 77.8 a 26.6, Lakota de 69.2 a 28.0 y Nubio kunuz de 47.7 a 16.6. Este comportamiento es coherente con las limitaciones del paradigma *Self-Refine* descritas por Madaan y cols. (2023): si la crítica procede del propio modelo y no existe una señal externa

de validación, la revisión puede reforzar errores iniciales en lugar de corregirlos. Aquí surge el efecto contrario, el problema se agrava porque el modelo puede usar sus propias reglas como árbitro, incluso cuando esas reglas no están suficientemente justificadas por los ejemplos de entrenamiento.

Los experimentos complementarios permiten matizar todavía más la respuesta a la pregunta de investigación. La dirección inversa, inglés $\rightarrow$ lengua desconocida, resulta considerablemente más difícil que la dirección principal lengua desconocida $\rightarrow$ inglés. Además, las tareas morfológicas y numéricas refuerzan esta conclusión: en Guazacapán Xinka todos los modelos obtienen solo 1/4 en Exact Match, y en Supyire ningún modelo alcanza Exact Match en las tareas numéricas. Los modelos identifican regularidades parciales, pero no siempre logran aplicarlas con la precisión formal necesaria.

En conjunto, los resultados muestran que los modelos de lenguaje sí poseen cierta capacidad metalingüística inducida por contexto: pueden extraer vocabulario, detectar regularidades morfológicas, formular hipótesis y aplicar algunas reglas a frases nuevas. No obstante, esa capacidad no alcanza el nivel humano de forma estable en ninguna configuración evaluada. Las estrategias de prompting influyen de manera significativa en el rendimiento: según el modelo y el puzzle, cambiar la metodología puede suponer diferencias de más de 30 puntos. Aunque los modelos actuales todavía presentan limitaciones importantes, especialmente en producción morfológica y revisión autónoma, los resultados sugieren que este tipo de enfoques sí puede ser útil para analizar lenguas de baja densidad de recursos en contextos donde solo se dispone de unos pocos ejemplos.

8. Trabajos futuros

Los resultados obtenidos en este trabajo abren varias líneas de continuación. Algunas están relacionadas con ampliar el conjunto experimental y otras con mejorar las estrategias de *prompting*, estudiar nuevas direcciones de traducción o incorporar señales externas de revisión. En esta sección se recogen las principales ampliaciones que podrían desarrollarse a partir del sistema implementado.

8.1. Ampliación del conjunto de puzzles

En este trabajo se han evaluado 11 puzzles de traducción en dirección lengua desconocida $\rightarrow$ inglés, pertenecientes a 10 familias lingüísticas distintas. Aunque esta selección permite cubrir cierta diversidad tipológica, el benchmark LINGUINI contiene 114 problemas de traducción, por lo que una ampliación natural sería ejecutar el sistema sobre un conjunto mucho mayor. Adicionalmente, una evaluación más amplia permitiría estudiar si las tendencias observadas se mantienen al variar la familia lingüística, el número de frases de entrenamiento o la complejidad morfológica del puzzle. En particular, sería interesante analizar puzzles con más de 20 frases de entrenamiento, ya que estrategias como M2 podrían beneficiarse especialmente de disponer de más pasos para construir y revisar el diccionario acumulado.

8.2. Ampliación de evaluación de la dirección inversa y exploración combinatoria de ejemplos

En cuanto a la dirección inversa, los experimentos en dirección inglés $\rightarrow$ lengua desconocida realizados en esta memoria son exploratorios: se limitan a tres puzzles y a la estrategia M1. Aun así, los resultados muestran una degradación clara respecto a la dirección lengua desconocida $\rightarrow$ inglés, lo que confirma que producir formas en una lengua no familiar es una tarea especialmente difícil. Una línea futura sería evaluar de forma sistemática el conjunto completo de puzzles `en2lang`, aplicando todas las metodologías M1–M5. Esto permitiría cuantificar con mayor rigor la brecha entre comprensión y producción, y comprobar si estrategias como M4, basada en inspiración humana, pueden reducir esa asimetría cuando el modelo debe generar morfología en la lengua desconocida.

En cuanto a la exploración combinatoria, la estrategia M2.2 se ha implementado como una prueba exploratoria sobre el puzzle de Hakhun. Los resultados muestran que la selección de ejemplos iniciales tiene un impacto muy alto cuando el subconjunto es pequeño: con $k = 2$, la diferencia entre la mejor y la peor combinación pasa de `chrF++=71.6` a `chrF++=8.5`. Al aumentar a $k = 3$, la desviación típica baja de forma clara, lo que sugiere que el aprendizaje se estabiliza con una mínima variedad de ejemplos. Como trabajo futuro, M2.2 podría extenderse al conjunto completo de puzzles, utilizando valores de `MAX_K=4` o `MAX_K=5` cuando

el coste computacional lo permita. Además, esta estrategia podría combinarse con técnicas de selección activa de ejemplos, de forma que el sistema no evalúe todas las combinaciones posibles, sino que priorice aquellas con mayor diversidad léxica, morfológica o sintáctica. Esto convertiría M2.2 en una herramienta más práctica para escenarios con presupuesto limitado de tokens.

8.3. Mejora de las estrategias de revisión

Los resultados de M5 muestran que la autocorrección sin retroalimentación externa puede empeorar respuestas que inicialmente eran buenas. El modelo revisa sus propias hipótesis, pero si esas hipótesis son incorrectas, la segunda vuelta puede reforzar el error en lugar de corregirlo. Esto ocurre especialmente en Llama-3.3-70B, donde varios primeros borradores de alta calidad se degradan de forma importante durante la revisión.

Una posible mejora sería incorporar una señal externa de validación. Por ejemplo, se podría diseñar un verificador de cobertura de morfemas que comprobara si cada elemento de la frase de entrada está justificado por alguna entrada del diccionario o regla gramatical. Otra posibilidad sería separar los roles de generador y revisor, usando un modelo para producir la primera respuesta y otro distinto para evaluarla. Estas variantes seguirían la idea general de *Self-Refine*, pero evitando que la única fuente de crítica sea el propio modelo que cometió el error.

8.4. Más protocolos de razonamiento humano

La estrategia M4 utiliza únicamente el razonamiento humano del puzzle de Hakhun como ejemplo de proceso. Aunque esta guía mejora algunos resultados, especialmente en Llama-3.3-70B, su utilidad puede depender de la similitud entre el puzzle guía y el puzzle objetivo. Un único protocolo humano no tiene por qué representar todas las formas posibles de resolver problemas lingüísticos.

Una línea futura interesante sería recopilar más razonamientos humanos sobre puzzles de distintas familias y tipologías: por ejemplo, una lengua aglutinante, una polisintética, una tonal o una con sistema de casos complejo. Con varios protocolos disponibles, el sistema podría seleccionar automáticamente el razonamiento-guía más parecido al puzzle objetivo, o incluso comparar si ciertos estilos de explicación humana ayudan más que otros.

8.5. Evaluación con nuevos modelos

Durante el desarrollo del trabajo han aparecido nuevos modelos relevantes que no han podido incluirse por limitaciones de tiempo y acceso. La evaluación de modelos más recientes, especialmente aquellos ajustados para tareas de razonamiento, permitiría comprobar si las tendencias observadas se mantienen o cambian con nuevas arquitecturas.

Sería especialmente interesante comprobar si el patrón observado en este trabajo, donde Llama-3.3-70B supera globalmente a GPT-OSS-120B, se reproduce con otros modelos de tamaño medio y grande. También podría realizarse una prueba preliminar con modelos accesibles en Google Colab, como Gemma (Gemma Team y cols., 2024), aprovechando que el

sistema experimental ya está preparado para cargar puzzles, generar prompts y recalcular métricas sin modificar su estructura principal.

8.6. Uso de salidas estructuradas

Una posible mejora del sistema sería sustituir parte del filtrado heurístico por mecanismos de *structured outputs*. Este enfoque consiste en pedir o forzar al modelo a generar la respuesta siguiendo un esquema predefinido, por ejemplo un objeto JSON con campos separados para el diccionario, las reglas gramaticales y las traducciones finales. De esta forma, el sistema no tendría que extraer las traducciones a partir de texto libre, sino leerlas directamente de un campo concreto. Esto reduciría la dependencia de filtros diseñados manualmente y haría el proceso más robusto frente a diferencias entre modelos. Sin embargo, su implementación requeriría adaptar las llamadas a las APIs y rediseñar los prompts para que todos los modelos produzcan una salida validable, por lo que se deja como línea de mejora futura.

Bibliografía

- AI@Meta. (2024). *Llama 3 Model Card*. <https://github.com/meta-llama/llama3>.
- Aycock, S., Stap, D., Wu, D., Monz, C., y Sima'an, K. (2025). Can LLMs Really Learn to Translate a Low-Resource Language from One Grammar Book? En *Proceedings of the 13th international conference on learning representations*. Descargado de <https://openreview.net/forum?id=aMBSY2ebPw>
- Bean, A. M., Hellsten, S., Mayne, H., Magomere, J., Chi, E. A., Chi, R., ... Kirk, H. R. (2024). LINGOLY: A Benchmark of Olympiad-Level Linguistic Reasoning Puzzles in Low-Resource and Extinct Languages. En *Advances in neural information processing systems (neurips 2024), track on datasets and benchmarks*. Descargado de <https://arxiv.org/abs/2406.06196>
- Bozhanov, B., y Derzhanski, I. (2013). Rosetta Stone Linguistic Problems. En *Proceedings of the fourth workshop on teaching nlp and cl* (pp. 1–8). Sofia, Bulgaria: Association for Computational Linguistics.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... others (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. Descargado de <https://arxiv.org/abs/2005.14165>
- Chi, N., Malchev, T., Kong, R., Chi, R., Huang, L., Chi, E., ... Radev, D. (2024). ModeLing: A Novel Dataset for Testing Linguistic Reasoning in Language Models. En *Proceedings of the 6th workshop on research in computational linguistic typology and multilingual nlp* (pp. 113–119). St. Julian's, Malta: Association for Computational Linguistics.
- Choudhary, M., Srivatsa, K. A., Aeron, G., Bhattacharya, A. R., Dinh, D. K. D., Hanif, I. A., ... Choudhary, M. (2025). UNVEILING: What Makes Linguistics Olympiad Puzzles Tricky for LLMs? *arXiv preprint arXiv:2508.11260*. Descargado de <https://arxiv.org/abs/2508.11260>
- Derzhanski, I., y Payne, T. E. (2010). The Linguistics Olympiads: Academic Competitions in Linguistics for Secondary School Students. En *Linguistics at school: Language awareness in primary and secondary education* (pp. 213–226). Cambridge University Press.
- Eberhard, D. M., Simons, G. F., y Fennig, C. D. (2020). *Ethnologue: Languages of the World*. Dallas, Texas: SIL International. Descargado de <https://www.ethnologue.com> (23rd ed.)
- Ericsson, K. A., y Simon, H. A. (1993). *Protocol Analysis: Verbal Reports as Data* (Revised edition ed.). Cambridge, MA: MIT Press.

- Gemma Team, Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., ... others (2024). Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295*. Descargado de <https://arxiv.org/abs/2403.08295>
- International Linguistics Olympiad. (2026). *Problems*. <https://ioling.org/problems/>. (Accedido en junio de 2026)
- Joshi, P., Santy, S., Budhiraja, A., Bali, K., y Choudhury, M. (2020). The State and Fate of Linguistic Diversity and Inclusion in the NLP World. En *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 6282–6293). doi: 10.18653/v1/2020.acl-main.560
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., ... Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. En *Advances in neural information processing systems*. Descargado de <https://openreview.net/forum?id=S37h0erQLB>
- Majmudar, N., y Filatova, E. (2025). Can LLMs Solve and Generate Linguistic Olympiad Puzzles? *arXiv preprint arXiv:2509.21820*. Descargado de <https://arxiv.org/abs/2509.21820>
- Papineni, K., Roukos, S., Ward, T., y Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. En *Proceedings of the 40th annual meeting of the association for computational linguistics* (pp. 311–318). doi: 10.3115/1073083.1073135
- Popović, M. (2015). chrF: Character n-gram F-score for Automatic MT Evaluation. En *Proceedings of the tenth workshop on statistical machine translation* (pp. 392–395). Lisbon, Portugal: Association for Computational Linguistics. doi: 10.18653/v1/W15-3049
- Popović, M. (2017). chrF++: Words Helping Character n-grams. En *Proceedings of the second conference on machine translation* (pp. 612–618). Copenhagen, Denmark: Association for Computational Linguistics. doi: 10.18653/v1/W17-4726
- Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. En *Proceedings of the third conference on machine translation: Research papers* (pp. 186–191). Association for Computational Linguistics. doi: 10.18653/v1/W18-6319
- Ramji, R., y Ramji, K. (2024). Inductive Linguistic Reasoning with Large Language Models. *arXiv preprint arXiv:2412.17819*. Descargado de <https://arxiv.org/abs/2412.17819>
- Snover, M., Dorr, B., Schwartz, R., Micciulla, L., y Makhoul, J. (2006). A Study of Translation Edit Rate with Targeted Human Annotation. En *Proceedings of the 7th conference of the association for machine translation in the americas: Technical papers* (pp. 223–231). Cambridge, Massachusetts, USA: Association for Machine Translation in the Americas. Descargado de <https://aclanthology.org/2006.amta-papers.25/>
- Sánchez, E., Alastruey, B., Ropers, C., Stenetorp, P., Artetxe, M., y Costa-jussà, M. R. (2024). Linguini: A Benchmark for Language-Agnostic Linguistic Reasoning. *arXiv preprint arXiv:2409.12126*. Descargado de <https://arxiv.org/abs/2409.12126>

- Tanzer, G., Suzgun, M., Visser, E., Jurafsky, D., y Melas-Kyriazi, L. (2024). A Benchmark for Learning to Translate a New Language from One Grammar Book. En *Proceedings of the 12th international conference on learning representations (iclr 2024)*. Descargado de <https://arxiv.org/abs/2309.16575>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35. Descargado de <https://arxiv.org/abs/2201.11903>
- Zhu, H., Liang, Y., Xu, W., y Xu, H. (2025). Evaluating Large Language Models for In-Context Learning of Linguistic Patterns In Unseen Low Resource Languages. En *Proceedings of the first workshop on language models for low-resource languages* (pp. 414–426). Abu Dhabi, United Arab Emirates: Association for Computational Linguistics. Descargado de <https://aclanthology.org/2025.loreslm-1.31/>
- Şahin, G. G., Kementchedjhieva, Y., Rust, P., y Gurevych, I. (2020). PuzzLing Machines: A Challenge on Learning From Small Data. En *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 1241–1254). Online: Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.115
-

A. Repositorio del proyecto

El código desarrollado durante este trabajo de fin de grado se encuentra disponible en el siguiente repositorio: <https://github.com/aliciavictoria/LINGUINI-LLM-PROMPTING>

El repositorio incluye los scripts principales del sistema experimental, el código de ejecución de las estrategias, el módulo de extracción y recálculo de métricas, así como las instrucciones necesarias para descargar el benchmark Linguini, configurar las claves de API y reproducir los experimentos descritos en la memoria.